GDAŃSK UNIVERSITY OF TECHNOLOGY

FACULTY OF ELECTRONICS, TELECOMMUNICATIONS AND INFORMATICS

Student’s name and surname: Kacper Narloch
ID: 175752
Cycle of studies: master’s degree studies
Mode of study: full-time studies
Field of study: Informatics
Specialization: Distributed Applications and Internet Services

MASTER'S THESIS

Title of the thesis: Implementation of PWA with Blazor technology

Title of the thesis (in Polish): Wykorzystanie technologii Blazor do realizacji aplikacji typu PWA

Supervisor: dr inż. Krzysztof Manuszewski

ABSTRACT

Progressive Web Applications (PWA) are single-page applications that behave like traditional desktop or mobile applications. Although typically developed using JavaScript-based frameworks, Blazor presents an alternative approach using .NET technology. Blazor allows developers to build interactive web applications using C# instead of JavaScript, potentially offering new advantages for PWA development. However, the specific benefits and limitations of using Blazor for PWAs have not been explored.

This study aims to fill this gap by evaluating two functionally identical applications, one developed using Blazor and the other with Vue.js, a widely adopted JavaScript framework. The development process for both applications is described in detail, highlighting the architectural decisions, implementation challenges, and specific techniques used to meet the requirements of PWA. The applications were compared by initial load time, runtime performance, and application size. In addition, the developer experience was measured by examining the volume of code and the startup time of the development server.

The findings show that Blazor with WebAssembly-based client-side rendering is a suitable option for developing PWAs, especially when performance is not the primary concern. It meets essential PWA requirements such as offline support, installability, and background synchronization. It also provides advantages in development efficiency and maintainability by reducing the volume of code needed for application logic. However, its larger application size and performance trade-offs make it less suitable for performance-critical applications compared to frameworks like Vue.js.

Keywords: Progressive Web Application, Blazor, Vue.js, WebAssembly, JavaScript frameworks, .NET
Field of science and technology in accordance with OECD requirements:
Computer and Information Sciences

CONTENTS

Acronyms . . . 6
1. Introduction . . . 7
   1.1. Context of the study . . . 7
   1.2. Goals of the Study . . . 8
   1.3. Outline of the thesis . . . 8
2. State of the art . . . 9
3. Progressive Web Application . . . 12
   3.1. Core technologies . . . 13
   3.2. Web Application Manifest . . . 13
   3.3. Service Workers . . . 14
      3.3.1. Lifecycle of service workers . . . 15
      3.3.2. Caching and offline functionality . . . 16
      3.3.3. Push notifications . . . 17
      3.3.4. Background sync . . . 18
4. Blazor . . . 19
   4.1. WebAssembly . . . 19
   4.2. Blazor WebAssembly . . . 20
      4.2.1. Project Startup . . . 20
      4.2.2. User interface updates . . . 20
5. Vue . . . 21
   5.1. Overview . . . 21
   5.2. Rendering mechanism . . . 21
   5.3. Build tools . . . 22
6. Development of experimental progressive web applications . . . 23
   6.1. PWA Features . . . 25
   6.2. Responsive design . . . 25
   6.3. Project Setup . . . 27
   6.4. Manifest . . . 28
   6.5. Service Worker . . . 31
      6.5.1. Workbox integration . . . 31
      6.5.2. Service worker implementation . . . 32
   6.6. Hosting . . . 35
7. Comparison of PWA Implementations . . . 36
   7.1. Lighthouse performance test . . . 36
   7.2. Application Size . . . 40
   7.3. Rendering performance . . . 41
   7.4. Volume of code . . . 43
      7.4.1. Presentation Layer . . . 43
      7.4.2. Infrastructure Layer . . . 44
      7.4.3. Domain Layer . . . 45

7.4.4. Application Startup 46
7.4.5. Overall Code Volume Analysis 46
7.5. Development server startup time 47
8. Conclusion 48
REFERENCES 51
LIST OF FIGURES 52
LIST OF TABLES 53

ACRONYMS

API Application Programming Interface. 10, 15, 17, 18, 21–23, 32, 35, 36, 40–42, 44, 45

CDN Content Delivery Network. 31, 32

CLI Command Line Interface. 27, 32, 35, 48

CPU Central Processing Unit. 16

CSR Client-Side Rendering. 19

CSS Cascading Style Sheets. 11, 20, 21, 31

DOM Document Object Model. 9–11, 20, 21, 41, 42, 46, 48, 49

ESM ECMAScript Modules. 22

HTML HyperText Markup Language. 13, 19–21, 31

HTTP Hypertext Transfer Protocol. 9, 10, 22

HTTPS Hypertext Transfer Protocol Secure. 13, 23, 25, 35

IDE Integrated Development Environment. 21

JSON JavaScript Object Notation. 13, 31

LINQ Language Integrated Query. 11, 45

LOC Lines of Code. 43–46

PWA Progressive Web Application. 7, 8, 12, 13, 15–19, 23, 25, 27, 28, 31, 36, 39, 40, 43, 46, 48, 49

REST Representational State Transfer. 17

SDK Software Development Kit. 27

SPA Single Page Application. 11

SSL Secure Sockets Layer. 35

SSR Server-Side Rendering. 19

UI User Interface. 10, 19, 20, 25, 30, 41, 42

URL Uniform Resource Locator. 13, 44

1. INTRODUCTION

1.1. Context of the study

Over the last decade, we have seen significant changes in the field of web development, driven by advances in web frameworks, increased computer performance and bandwidth, and shifting user expectations. One of them started in 2015, when two Google employees, developer Alex Russell and designer Frances Berriman, introduced the concept of Progressive Web Applications (PWAs). They saw PWAs as a way to bring the best user experiences from native mobile apps to the web by using features such as service workers and web app manifests. [1]

The usage and popularity of the new web application standard have increased significantly since it was introduced. Flipkart, India's largest e-commerce platform, was one of the first to use this technology in their application, Flipkart Lite. This new application dramatically improved the user experience compared to the previous mobile platform. The adoption of the new standard resulted in users spending three times more time on the website, a 40% higher reengagement rate, a 70% higher conversion rate among users who reached the site using the "Add to Homescreen" function, and a three times lower data usage.[2]

Other major companies have recognized the value of PWAs and have joined the trend. Platforms such as Twitter, Forbes, Alibaba, and many others benefited significantly, with lower data consumption, significant increases in conversions, monthly active users, as well as faster loading times and higher user sessions and engagement. [1]

The advantages of PWAs are not limited to large corporations, as they can offer significant benefits to startups by accelerating product development and reducing costs. Unlike traditional native apps, they can be developed, tested, and deployed quickly, allowing startups to iterate through ideas cost-effectively. This makes them a powerful tool for companies that want to accelerate the development of prototypes and effectively match product to market. Using PWAs, startups can allocate resources more efficiently to innovation and refinement of their offerings, rather than investing heavily in native app development. [3]

Given the numerous benefits provided by PWA, the technology has been widely adopted since its introduction in 2015. However, when deciding to adopt this web standard, developers face a crucial question: which framework and programming language to use? Traditionally, JavaScript has been the main language for web development, and frameworks such as Angular, React, and Vue.js were designed to facilitate PWA creation. However, the web development environment is changing and new technologies are emerging that can provide attractive alternatives.

This thesis examines Blazor technology as one of the alternatives to JavaScript frameworks in PWA development. Blazor, a technology developed by Microsoft, is a modern front-end web framework that allows developers to create interactive web applications using mainly C# and .NET, while still allowing for integration with JavaScript. It supports two hosting models: server-side and WebAssembly. The server-side model performs application logic on the server and handles the user interface via a real-time SignalR connection, whereas Blazor WebAssembly runs code client-side in the browser on a WebAssembly-based .NET runtime. [4]

1.2. Goals of the Study

The purpose of this study is to investigate the suitability of using Blazor technology to implement progressive web applications. The study covers the implementation of selected aspects of PWA in Blazor and compares them with a reference technology built on JavaScript, in particular Vue.js. To achieve this goal, the study answers the following research questions:

RQ1 What are the technical capabilities of Blazor in meeting PWA requirements?

RQ2 What are the advantages and disadvantages of using Blazor to build PWAs in comparison to JavaScript-based technologies?

1.3. Outline of the thesis

In the following, all the chapters included in the thesis are listed, together with a brief description:

1. **Introduction** - Provides background information on the thesis and its objectives.
2. **State of the Art** - Reviews existing literature on Blazor in the context of modern web development, identifying a gap in studies on its use for Progressive Web Applications and comprehensive evaluations of developer experience.
3. **Progressive Web Application** - Introduces the concept of Progressive Web Applications and covers essential PWA technologies such as the Web App Manifest, Service Workers, and HTTPS.
4. **Blazor** - Provides an in-depth look at the Blazor framework, particularly focusing on Blazor WebAssembly, which enables PWA implementation. Describes how Blazor allows C# development in the browser and explores the structure and behavior of Blazor applications.
5. **Vue** - Presents Vue.js, chosen as the reference technology to compare how Blazor differs from a JavaScript framework in the context of PWA development. It discusses Vue's rendering mechanisms, component system, and supporting build tools.
6. **Development of Experimental Progressive Web Applications** - Describes the implementation of two equivalent PWA applications, one developed using Blazor and the other using Vue. Details the features, responsive design, service worker setup, and hosting configurations of both applications.
7. **Comparison of PWA Implementations** - Presents an analysis of the two applications in terms of initial load and runtime performance, size, volume of code, and development server startup time.
8. **Conclusion** - Summarizes the findings of the study, reflecting on the strengths and limitations of Blazor for the development of PWA and outlines recommendations for future work.

2. STATE OF THE ART

In the thesis **“WebAssembly Approach to Client-side Web Development using Blazor Framework“**[5] author compared the three most popular JavaScript frameworks to Blazor technology. The comparison was performed by evaluating the learning curve of each technology, the size needed to be downloaded to the browser to ensure proper functionality of this framework, and the existence of developer tools to support the creation of the application. According to the findings, the Blazor framework was moderately challenging to learn due to its architectural concepts, but easier to use than Angular, while it was more difficult than Vue.js. In terms of application size, Blazor got the worst score, requiring approximately five times more memory than React and Vue.js. A similar score was reached in comparison to ram usage, where Blazor was also ranked last. However, it is important to note that the tests were conducted using a preview version of Blazor, and in the future, it may achieve better results and become a popular choice for web application development.

The thesis is not focused solely on comparing frameworks, dedicating only one chapter in which we will find a description of the achievements of the above-mentioned technologies. In the further part, the authors present detailed information about the architecture of Blazor Server and Blazor WebAssembly applications and describe how the development process in this technology looks like.

In the article **"WebAssembly as an alternative to JavaScript in the development of modern web applications"** [6], Blazor technology was used to investigate the potential of WebAssembly technology in the development of web applications. As a benchmark for evaluating the performance of the Blazor application, a twin version of the application was developed using Angular. The comparisons involved various aspects such as application loading time, data transfer size, duration of Hypertext Transfer Protocol (HTTP) requests handling, and difficulty of writing code. In addition, two separate implementations were prepared for testing purposes, focusing on measuring the execution time of sorting object lists and the generation of Document Object Model (DOM) elements.

The results showed that the Blazor performed worse than the Angular in almost all criteria tested. The size of the Blazor application was significantly larger than that of the Angular application. This is due to the need to download the runtime environment, however, these data do not need to be downloaded every time the application loads since web browsers store them in a cache, significantly improving performance. As the initial loading time is influenced by the size of the application and the number of requests, this metric was also favorable for a JavaScript-based framework. Furthermore, the results revealed that Blazor experienced longer delays in handling HTTP requests compared to Angular, as WebAssembly currently needs more direct HTTP operation capabilities.

Another experiment compared the performance of JavaScript, C++, Angular, and Blazor in sorting an n-element list of objects. The results revealed a significant performance difference between JavaScript and WebAssembly, with C++ compiled into WASM performing operations significantly faster than equivalent JavaScript code. The performance of Angular remained comparable to that of a single script. However, regardless of collection size, Blazor performed significantly worse than Angular in the sorting test.

The authors also measured the performance of modifying the DOM elements in both applications. Compared to Angular, Blazor took much longer to add a thousand, three thousand, and ten thousand elements to the document tree. Because WebAssembly currently lacks direct access to the DOM tree, all Blazor modifications are carried out using generated JavaScript code, so delays were expected,

similar to the HTTP operations. The code metrics analyzed do not definitively indicate which framework requires less effort to implement the same functionality. In terms of the logic layer, five Blazor components had fewer lines of code than the remaining five Angular components. Angular had fewer lines of code in seven components in the presentation layer, while both frameworks had one component with the same number of lines. It implies that the syntax of Angular may be more precise, resulting in more condensed code.

The authors assessed that Blazor demonstrates the potential of WebAssembly by enabling the creation of functional single-page applications using a different approach to traditional JavaScript coding. Although WebAssembly can be an alternative to JavaScript, it does not replace it entirely, as certain browser functionalities can only be achieved through JavaScript.

Article **"Modern Web Frameworks: A Comparison of Rendering Performance"**[7] describes the rendering strategies used in Angular, React, Vue, Svelte and Blazor, which are the most popular web frameworks. All reviewed frameworks share a common goal: maintaining synchronization between the state of DOM and the state of the component tree by creating and updating the DOM nodes and their attributes.

The authors conducted various different benchmarks, and in each one they implemented an application identically with each framework and performed a particular action that measures the costs of a particular aspect of rendering by varying the size and shape of the component tree. After each scenario, they meticulously measure the time taken for script execution in the render cycle. The benchmark tests performed are used to compare the frameworks in terms of relative differences. The results are presented in absolute terms, measured in milliseconds, but the authors point out that the simplicity of the components used does not accurately reflect the expected performance of a real application with a component tree of equivalent size.

The results show that Blazor, being the only WebAssembly-based framework among the reviewed ones (Angular, React, Vue, and Svelte), exhibits significantly worse performance compared to its JavaScript-based counterparts, including React, which shares an identical rendering model. It is noted that this discrepancy in performance cannot definitively be attributed to factors specific to Blazor or inherent weaknesses in WebAssembly itself. However, the need to mediate access to DOM Application Programming Interface (API) through a JavaScript layer is identified as a factor contributing to the observed overhead in Blazor's results.

Furthermore, the results indicate that frameworks such as Blazor, which lack the capability to automatically determine dirty components as provided by frameworks like Vue.js, experience performance challenges. This can particularly affect small transitions and complex User Interfaces (UIs), which can lead to an unresponsive user experience. It is implied that such frameworks may require manual optimization efforts, which are automatically handled by other frameworks.

In the article **"Analysis of the Blazor framework in client-hosted mode"**[8] the authors conducted several experiments aimed to evaluate the performance of the Blazor framework in client-side mode with hosting, which are:

- measuring application loading performance, specifically the time taken for HTML loading, blocking requests (such as Cascading Style Sheets (CSS) and JavaScript), and complete page loading. The experiment involved a component displaying 10 cards with images
- calculating the value of PI using different precisions: 1000, 5000, and 10000 decimal places. The implementation used C# for the Blazor framework and JavaScript with the necessary adjustments to handle unsigned numeric types.
- calculating the Ackermann function between the Blazor framework and the JavaScript implementation.
- measuring the performance of Language Integrated Query (LINQ) operations in collections. The task involved finding the maximum and minimum even numbers in collections of various sizes, which were randomly populated with integers.
- measuring the performance of updating DOM for both Blazor in client-side mode with hosting and JavaScript.

A Razor component was developed for the Blazor application, which generates a table with a specified number of rows. The same functionality was implemented using JavaScript, which had direct access to the structure of DOM. The results showed that the first full page load time (Finish parameter) is approximately 10 times longer than the initial page load time (Load parameter). However, caching in the browser reduced subsequent application load times. In terms of computing PI, Blazor outperformed JavaScript as precision increased. However, when it came to the Ackermann function, JavaScript exhibited superior performance compared to Blazor, especially with more complex function input.

In the case of LINQ operations on collections, the study found that LINQ showed good scalability, efficiently determining minimum and maximum values. Lastly, JavaScript proved to be more efficient than Blazor in modifying the DOM, particularly when dealing with a larger number of elements.

The authors note good performance in collection operations and computational scenarios, but mention initial application size and optimization issues. They also highlight limitations in handling recursive functions and generating DOM structures. Despite these drawbacks, they see potential in Blazor's use of WebAssembly and anticipate future performance improvements.

The existing literature focuses mainly on factors related to Single Page Applications (SPAs) and their development using Blazor. Nevertheless, the literature review lacks publications exploring the possibilities and advantages of employing the Blazor framework for developing Progressive Web Applications. PWAs are increasingly recognized as a contemporary approach to web application development, so investigating the potential of Blazor within this realm could offer valuable insights and perspectives to developers working with the .Net stack.

Furthermore, most existing studies focus mainly on performance metrics, offering only limited insight into the developer experience. Evaluations that utilize code metrics analyze the number of lines of code within components, which may not accurately reflect the effort involved in the development process. This highlights the need for a more in-depth assessment of the developer experience.

All reviewed publications are approximately three years old. Since then, the Blazor framework has undergone significant advances and improvements. This provides an opportunity to provide a more up-to-date understanding of Blazor's suitability for different application development scenarios.

3. PROGRESSIVE WEB APPLICATION

The universal accessibility, strong security, and open standards of the Web make it an ideal platform to deliver software that anyone can access and share. Web applications work on any device with a single codebase and are easy to deploy without the need for packaging, content reviews, or dealing with update delays. Users are guaranteed to get the most recent version every time they access an application.[9]

In contrast, platform-specific applications offer deep integration with devices, enabling rich functionality and offline use. However, they are often tied to specific platforms, making cross-platform compatibility challenging. **Progressive Web Applications** merge the strengths of both, combining the accessibility of web applications with the user experience similar to native applications.[9]

Progressive Web Applications (PWAs) are called "progressive" because they rely on a technique known as progressive enhancement. This approach focuses on developing a core experience that works across all devices and browsers, while allowing for more advanced functionality on more capable devices. With progressive enhancement, every user has access to the application's core functionality, regardless of device limitations. However, as the device and browser engine's capabilities increase, the application will be able to unlock new features and provide better performance. On more advanced devices, PWA may offer offline access, push notifications, or integration with hardware components such as the camera or GPS. [10]

Progressive enhancement is an important characteristic of Progressive Web Applications, but there are more factors that contribute to delivering a native app-like experience. Google identified five essential criteria that also play a significant role in making a web application installable, reliable, and user-friendly on different devices and browsers.[11]
Key aspects include:

- **Performance:** A PWA should load quickly and maintain high performance throughout the user interaction. Fast loading is critical because slow page loads significantly increase the likelihood of user abandonment. Consistently smooth performance, including responsive interactions and animations, enhances the user experience.
- **Cross-browser compatibility:** PWAs are fundamentally web applications, which means they must function across all browsers. A practical approach to achieving this is to first identify the core features and implement them using the most basic technology available. Once this foundational functionality is established, additional features and improvements can then be added.
- **Responsive design:** It is important to note that devices vary in size and that users may interact with an application on different screens. While the tasks that users need to complete and the content that they want to access remain constant, it is necessary to adjust and rearrange the content to ensure that it is accessible and functional on any viewport size.
- **Custom Offline Page:** Instead of displaying the browser's default offline page, a PWA should provide a custom offline page to maintain a consistent user experience when the user is offline.
- **Installability:** PWAs should be installable, allowing users to add the application to their device's home screen. This installation mimics the behavior of native apps, leading to increased engagement and improved user metrics such as repeat visits and longer sessions.[11]

3.1. Core technologies

Although a PWA may include many different features, there are three technologies that are essential for an application to be considered a true progressive web application:

- **Web Application Manifest**: This file is critical to making PWAs discoverable and installable. It contains important metadata, including the name of the application, launch Uniform Resource Locator (URL), icons, and other details needed to present the website as an app on users' devices.
- **Service Worker**: This is a technology that enables developers to intercept and manage network requests and asset caching within the web browser. Service workers are key to creating fast, reliable web pages and enabling offline functionality, which are hallmarks of PWAs.
- **Hypertext Transfer Protocol Secure (HTTPS)**: PWAs must be delivered over a secure server using HTTPS. The security provided by HTTPS is critical because many of the advanced features of PWAs, particularly the service workers, are only available over secure connections. [10]

3.2. Web Application Manifest

A Web Application Manifest is a JavaScript Object Notation (JSON) file that provides metadata about a web application. This file is used by web browsers to allow users to install web applications on their devices. The manifest file allows the developer to define how their web app should appear to the user and provides the information necessary for adding it to the user's home screen.

The manifest file is linked to the HyperText Markup Language (HTML) of the web app via a <link> tag situated within the <head> section of the document, as shown in the Listing 3.1. [12]

Listing 3.1: Usage of manifest.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="manifest" href="manifest.json" />
  </head>
  <body></body>
</html>
```

According to the manifest specification, all fields are optional. However, to meet the requirements for Progressive Web Apps, certain manifest members must be included, such as:

- `name` or `short_name`
- `icons` must contain a 192px and a 512px icon
- `start_url`
- `display` and/or `display_override`
- `prefer_related_applications` must be false or not present [12]

The example manifest file is presented in the Listing 3.2.

Listing 3.2: The example of a web application manifest

```
{
  "name": "DartPointTracker",
  "short_name": "DartPointTracker",
  "id": "./",
  "start_url": "./",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#03173d",
  "prefer_related_applications": false,
  "icons": [
    {
      "src": "icon-512.png",
      "type": "image/png",
      "sizes": "512x512"
    },
    {
      "src": "icon-192.png",
      "type": "image/png",
      "sizes": "192x192"
    }
  ]
}
```

3.3. Service Workers

Service workers are a technology that brings multiple capabilities to web applications. The most significant of these is the ability to operate in scenarios where Internet connectivity is either limited or unavailable. They function as proxy servers, situated between the web application, the browser, and the network. This enables the app to perform tasks such as offline access, interception of network requests, caching of resources, and providing fallback content when the network is not accessible. [13]

A service worker is an event-driven background task, created via a script, that operates within a specific origin (the combination of protocol, domain, and port) and a defined path, which determines the scope of its control. It has the form of a JavaScript file that manages the associated web page by intercepting and modifying navigation and resource requests, and caching resources. When an app requests a resource, it can serve it from the cache, fetch it from the network, or generate it locally, providing a platform-like experience even when offline. Running in a separate worker context, service workers are non-blocking, fully asynchronous, and operate on a different thread from the main app, ensuring they don't interfere with primary processes.[13] [14]

3.3.1. Lifecycle of service workers

In order to successfully implement service workers in PWAs, it is essential to gain an understanding of their lifecycle. This involves a number of distinct phases, which are illustrated in the Figure 3.1.

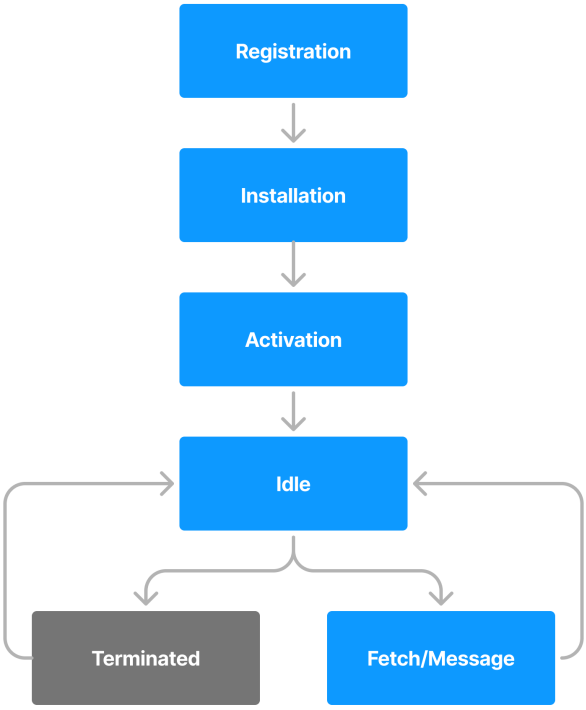

Figure 3.1: The lifecycle of service worker

Registration

The journey of a service worker begins with registration. When a user first opens the website, all network requests initially go directly to the server because the service worker hasn't taken control yet. The registration process starts by checking if the browser supports the Service Worker API. Once this is confirmed, PWA registers the service worker script. The location of the script determines its scope; placing it at the root gives it control over the entire domain, while placing it in a subdirectory limits its scope. [14]

Installation

Once it is registered successfully, the installation phase begins. This process happens silently in the background, without needing user permission, even if the user does not install PWA.If the installation fails, the service worker does not take control of the page. The successful installation fires the install event, which occurs only once per service worker. Developers can listen for that event to prepare the service worker for use, typically by creating a cache and storing static assets needed for offline functionality.[15]

Activation

After successful registration and installation, the service worker state changes to 'activating'. During the activation, tasks such as cleaning up old caches can be performed, which is important for service worker updates. For a new service worker, activation occurs immediately after installation. Once activation is complete, the state changes to 'activated'. By default, a new service worker will only take control of the pages on the next navigation or page refresh. This ensures that any currently active pages continue to function under the control of the previous service worker until they are reloaded. [16]

Termination or Action

Service workers are designed to be resource efficient, which means that they run only when necessary. After performing their tasks, service workers enter an idle state. If there are no active clients within their scope, the service worker will be terminated by the browser after being idle for a few seconds or working too long, to conserve memory and Central Processing Unit (CPU) resources. However, service workers are automatically restarted when needed, such as when a new network request occurs or a message is sent to them. This ensures that service workers are available to perform necessary actions without consuming resources unnecessarily.[17]

3.3.2. Caching and offline functionality

Caching is a fundamental aspect of developing PWAs that enables them to be both fast and reliable and less dependent on network conditions. Service workers allow developers to cache specific assets independently of the main application thread, providing greater control over how and when resources are stored on users' devices. However, this also brings with it the responsibility to use storage space and bandwidth wisely, as these assets are downloaded and stored locally. The key is to achieve an optimal balance between caching enough assets to ensure a fast, reliable, or offline experience without overloading the user's device with excessive data. [18]

To achieve fast load times, it is essential to cache the components required to render the user interface. This typically includes the main page's HTML, CSS stylesheets, images, JavaScript files, web fonts, etc. By caching these assets, PWA is able to rapidly display a fully functional interface. This process typically occurs during the Service Worker's installation. [18]

In addition to these assets, providing an offline experience requires caching at least an offline page. This page informs users that they are offline and that certain requests cannot be completed at the moment. However, the offline experience can be further enhanced by caching interactive elements, allowing users to continue engaging with the app even without a network connection, for example, viewing previously accessed content.This is possible by intercepting network requests in service workers on a fetch event.[10]

With the fetch event, there are different strategies for handling user requests:

- **Cache First:** Resources are retrieved from the cache first, ensuring quick load times and a network fallback in case there is no matching request in the cache.
- **Network First:** The strategy prioritizes the retrieval of the latest content from the network, making it ideal for frequently updated resources like articles and social media feeds, with a fallback to cached content in the event of a network failure. However, this approach can result in delays for users with slow or unreliable connections, as they must wait for the network request to time out before accessing cached content, which could lead to a frustrating experience.

- **Stale-While-Revalidate:** The strategy serves cached content while simultaneously fetching updated content from the network, which is then stored in the cache for future use. This approach ensures that users have fast access to cached resources while keeping the cache up-to-date with the latest content in the background. Ideal for resources that change very often but where the newest version is not essential.[19]

3.3.3. Push notifications

Another feature enabled by the service workers in PWAs that allows them to function like native applications is the ability to send push notifications. The automatic delivery of push messages and notifications helps with re-engaging users and delivering new content as soon as it becomes available.

The implementation of push notifications is achieved through the collaboration of two distinct APIs: the Push API and the Notifications API. The Push API enables web applications to receive messages sent from a server, regardless of whether the application is in the foreground or even currently loaded on a user agent. This enables developers to deliver asynchronous notifications and updates to users who have consented to receive them. The Notifications API allows web pages to manage the display of system notifications to the user. These notifications appear outside the main browsing context, which means that they can be shown even when the user has switched tabs or moved to another app. [20]

To successfully deliver push notifications to the user, it is essential to understand the three key steps involved in the process:

1. **Client-Side Subscription:** The first step is to subscribe the user to receive push notifications. This involves asking the user for permission through the Notifications and Permissions APIs and then using the Push API to deliver a *PushSubscription*. This is an object that contains all the necessary information to send a push message to the user's device, including, inter alia, an endpoint pointing to the user's browser push service and the keys to encrypt the push message data. Once *PushSubscription* is obtained, it is sent to the backend server and stored in a database for future use.
2. **Sending a Push Message:** When the server wants to send a push message, it makes a Representational State Transfer (REST) API call to the endpoint of a push service. This push service is responsible for handling the request, managing the delivery of the message, and queueing it if the device is offline. The API call includes details such as the recipient's information, the data to be sent, and specific instructions on how the message should be delivered, such as time-to-live and urgency levels. The push message data is encrypted, preventing push services from viewing it. This is a crucial aspect, as the browser determines which push service is used, which could potentially result in browsers employing a push service that is not secure.
3. **Handling the Push Event:** Once the push service successfully delivers the message, the browser on the user's device receives it, decrypts any data, and triggers a push event in the service worker. The service worker may then perform a variety of tasks, including displaying a notification to the user, making analytics calls, or caching content for offline use.[21]

3.3.4. Background sync

PWAs are designed to function effectively even with unstable internet connectivity, so they need a mechanism to handle situations where users must interact with a server. Background Sync offers a solution that allows tasks to automatically resume when connectivity is restored after a failed API call due to poor network conditions.

A chat app serves as a great example of utilizing this feature, where the user wants to ensure that their message is delivered. In areas with unreliable connectivity, a regular web app might display a blank screen, a loading spinner, or an error message when trying to send a message. However, with Background Sync in PWA, you can still compose and send messages while offline. The app will queue these messages and automatically send them once the connection is reestablished.[22]

The implementation of Background Sync in an application begins by registering a sync event with a unique tag as shown in the Listing 3.3. The tag serves to identify the specific task that needs to be executed. [23]

Listing 3.3: Sync event registration.

```
navigator.serviceWorker.ready.then(function(swRegistration) {
  return swRegistration.sync.register('myFirstSync');
});
```

Subsequently, the code to listen for the sync event must be added to the service worker. As demonstrated in Listing 3.4, when the sync event is triggered, the tag is checked, and if it matches, the doSomeStuff() action is attempted. If the network is available, the sync event will complete successfully; if not, it will automatically retry using an exponential back-off strategy until it succeeds. [23]

Listing 3.4: Listening for sync event.

```
self.addEventListener('sync', function(event) {
  if (event.tag == 'myFirstSync') {
    event.waitUntil(doSomeStuff());
  }
});
```

4. BLAZOR

Blazor is an open-source web framework developed by Microsoft as part of the .NET platform. It has been designed to enable the creation of interactive web applications using C# and .NET Core as an alternative to JavaScript for front-end development. Blazor applications are constructed with components, which are fundamental UI elements written as .NET C# classes that define dynamic UI rendering logic and manage user events. Typically, they are implemented using a Razor syntax that combines C# and HTML markup in files with the .razor extension. [24]

Blazor is operating with two rendering modes: Client-Side Rendering (CSR) and interactive Server-Side Rendering (SSR). In SSR, user interactions and UI updates are handled through a real-time connection to the browser, established via SignalR. This technology employs WebSockets to transmit events and UI changes between the client and the server. In CSR, WebAssembly is used to run C# code directly in the user's browser, allowing each user to have their own local instance of the app. In contrast, Blazor Server operates with a single instance of the application on the server, to which multiple users connect simultaneously.[25]

With the release of .NET 8 in 2023, Blazor introduced interactive Auto (automatic) rendering, which enables the use of both server-side and client-side rendering within a single web application. Initially, the server-side ASP.NET Core runtime handles content rendering and interactivity. Once the Blazor bundle is downloaded and the WebAssembly runtime is activated on the client, subsequent rendering and interactivity shift to the client side. This approach typically offers the fastest app startup experience.[26]

However, creating Progressive Web Applications with Blazor is only feasible by developing applications that run entirely on WebAssembly in the browser, without involving a server. Server-side rendering requires a stable network connection to maintain an active session between the user and the server, which conflicts with the offline functionality of PWAs. The standalone Blazor WebAssembly apps deploy assets as static files to a web server, and once downloaded, they can be cached and run offline as PWAs.[24]

4.1. WebAssembly

WebAssembly is a type of code designed to run in modern web browsers, offering significant performance enhancements. In contrast to JavaScript, which is the dominant language in web development, WebAssembly is not typically created manually. Instead, it functions as a compilation target for languages such as C++ or C# and others. This allows developers to execute code written in these languages on the web at near-native speeds, enabling the creation of high-performance web applications that were previously impractical due to the limitations of JavaScript. [27]

As a high-level language, JavaScript can be resource-intensive to download, parse, and compile, which may lead to challenges in delivering a smooth user experience on devices with limited computing power, especially for more demanding use cases. WebAssembly uses binary format files which are smaller than equivalent files containing JavaScript code, reducing the time required for network downloads. Additionally, because the WebAssembly code is precompiled, it can be executed directly by a browser. These features lead to improved performance, enabling faster loading and execution. As a result, WebAssembly enables the development of advanced web applications such as 3D games,

virtual and augmented reality, complex image and video processing, and more.[27]

WebAssembly is platform independent, which means it will run the same code on any device with a compatible browser, regardless of operating system or hardware architecture. It runs in a secure, sandboxed environment separate from the host machine. This reduces the possibility of malicious code interacting with the system outside its intended boundaries.[27]

4.2. Blazor WebAssembly

4.2.1. Project Startup

To understand how the applications written in C#, after compilation into WebAssembly, are fully executed in a browser, it is important to know the initial process behind starting the Blazor WebAssembly application.

The startup process begins with the initial page load. The browser sends a GET request to the server to fetch the main page of the Blazor WebAssembly application, which is typically a static index.html file. The browser then parses HTML and loads linked resources such as CSS, JavaScript, and images. During this process, the browser initially renders the static HTML, displaying a placeholder, where the Blazor application will later render its content.[28]

As the Blazor WebAssembly initialization begins, the *blazor.webassembly.js* script is loaded and executed. This JavaScript file handles the loading and initialization of the Blazor application within the browser. First, it downloads the configuration file, *blazor.boot.json* , which contains a list of all required framework and application files. These include .NET assemblies and the *dotnet.wasm* file, which is a complete .NET runtime compiled into WebAssembly binary format. After downloading all the listed resources, the WebAssembly .NET runtime is initialized, loading the Blazor framework and the application. At this point, the application runs entirely in the client's browser. [28]

4.2.2. User interface updates

The role of the *blazor.webassembly.js* script does not end with the loading and initialization of the Blazor application. This part of the framework is crucial for performing the necessary DOM updates and providing APIs for scenarios requiring JavaScript interoperability. The use of JavaScript in these scenarios is necessary due to WebAssembly's limitations, as it cannot directly manipulate the DOM or interact with Web APIs. Although future advancements in WebAssembly may reduce the dependency on this file, at this moment it is essential for UI changes. [25]

When a user initiates a UI update, such as navigating from one page to another, the event is intercepted by Blazor's JavaScript runtime (blazor.webassembly.js), which communicates with the Blazor framework running in the WebAssembly runtime (dotnet.wasm). The Blazor router processes the navigation request by identifying the corresponding component, creating an instance of it, and executing its lifecycle methods. [25]

During this process, Blazor constructs a Render Tree, which serves as an internal representation of the UI structure of the component. To optimize updates, Blazor employs an incremental DOM approach. Instead of re-rendering the entire UI, it compares the newly generated Render Tree with the previous one to determine the minimal set of required changes. This comparison results in a differential render tree that identifies elements that need to be added, updated, or removed. The calculated updates are then sent back to the JavaScript runtime, which applies them to the actual DOM. [29]

5. VUE

5.1. Overview

Vue.js was selected as the reference technology to examine how Blazor differs from a widely used JavaScript framework in the development, performance, and capabilities of Progressive Web Applications. Released by software developer Evan You in 2014, Vue.js is an open-source framework that has gained significant popularity. Built on standard HTML, CSS, and JavaScript, Vue.js offers a declarative, component-based programming model that enables the efficient development of user interfaces of any complexity.[30]

Vue.js uses a template syntax that extends standard HTML that declaratively describes the HTML output based on the application's JavaScript state. The framework's reactivity system automatically tracks changes in the state and efficiently updates the Document Object Model when modifications occur. Similarly to Blazor's use of .razor files, Vue.js encapsulates each component within a file with .vue extension, which contains the logic, template, and styles in a single unit.[30]

Components can be written in two different API styles: Options API and Composition API. The Options API, which was the original way to define components, organizes component code by options such as data, methods, and computed properties. The Composition API, introduced in Vue 3, provides an alternative way of defining component logic using functions, which can be more suitable for organizing complex logic and promoting code reuse.[30]

Vue.js, like other popular JavaScript frameworks, supports TypeScript. TypeScript is a strongly typed programming language that builds on JavaScript and helps detect common errors through static analysis at build time, reducing the likelihood of runtime errors in production. It also makes refactoring code in larger applications more reliable and enhances the developer experience with type-based auto completion in Integrated Development Environment (IDE).[31]

5.2. Rendering mechanism

Vue.js rendering mechanism utilizes the Virtual DOM concept, which serves as an in-memory representation of the UI. Instead of inefficiently manipulating the real DOM directly, Vue.js creates lightweight JavaScript objects called virtual nodes that mirror the structure of actual DOM elements. When the application's state changes, Vue.js re-renders the Virtual DOM to produce a new virtual tree. Then it efficiently compares this new tree with the previous one to identify the minimal set of changes needed. This process allows Vue.js to update only the parts of the real DOM that have changed, significantly optimizing performance.[32]

5.3. Build tools

Vue.js currently uses Vite as its default build tool. Vite is a modern build tool developed by the Vue team, designed to provide a faster and more efficient development experience, offering an optimized workflow for both development and production builds. Unlike traditional bundlers, Vite takes advantage of native ECMAScript Modules (ESM), making it possible to deliver an optimized workflow for development builds.[33]

Vite improves the development phase by enhancing dev server start times and optimizing module handling. It separates modules into dependencies and source code: dependencies are pre-bundled using the fast "esbuild" tool since they change infrequently, accelerating initial load times. The source code is served over native ESM, allowing the browser to handle module loading and transform code on demand, reducing unnecessary processing. Vite enhances Hot Module Replacement by precisely updating only the changed modules, keeping updates fast regardless of application size and preserving application state without full page reloads. It also utilizes browser caching through HTTP headers to minimize server requests during reloads. By offloading tasks to the browser and minimizing processing, Vite maintains high performance even as the application grows, resulting in a faster and more efficient development experience.[33]

For the production build, bundling is still used because unbundled ESM can be inefficient due to extra network round trips from nested imports, even with HTTP/2. Bundling optimizes performance through tree-shaking, lazy-loading, and chunk splitting for better caching. Vite uses Rollup[1] for this process because it offers a more flexible plugin API, which fits Vite's infrastructure better than esbuild, despite esbuild's faster performance.[33]

[1] Rollup is a module bundler for JavaScript

6. DEVELOPMENT OF EXPERIMENTAL PROGRESSIVE WEB APPLICATIONS

The goal of this chapter is to explore the development of two Progressive Web Applications, both with identical functionality but built using different technologies. The first application was developed using Blazor, a .NET 8 based web development framework, while the second was created using Vue.js, a popular front-end framework in its latest stable version, Vue 3.4.29. The main objective is to evaluate the process of PWA development in Blazor, while using Vue.js as a comparative benchmark, as it is widely adopted in the web development community.

The **DartPointTracker** project is an application designed to assist players in recording and calculating scores during a darts game. Darts is a game in which players aim to reduce their score to exactly zero by hitting designated areas of the dartboard, each throw contributing to the player's total points. DartPointTracker simplifies scorekeeping and provides an automated system that calculates remaining points based on the scores input by the players. Beyond its scorekeeping function, the application also incorporates an Elo ranking system[1]. Each newly created player is assigned a default Elo ranking of 1000. After each game, the ranking of the players is adjusted based on their performance, with winners gaining Elo points and losers losing points.

The DartPointTracker requires the PWA to communicate with a backend web server that processes game results, manages players, and calculates Elo scores. This is accomplished through an API, where the PWA sends and receives data through HTTPS, and a database is used to store player information. The shared architecture across both the Blazor and Vue.js applications ensures that any differences between the two implementations are due to the technologies themselves, rather than differences in overall design. The architecture is presented in Figure6.1

Figure 6.1: Architecture of applications

Both Blazor and Vue.js are built on a component-based architecture, which means that the DartPointTracker applications share a modular structure across both implementations. In both frameworks, the user interface is divided into self-contained components, each responsible for a specific part of the application's functionality. This shared structure is visually represented in Figure6.2, where the relationship between each component is shown.

[1] Rating system to calculate the relative skill level of players in games

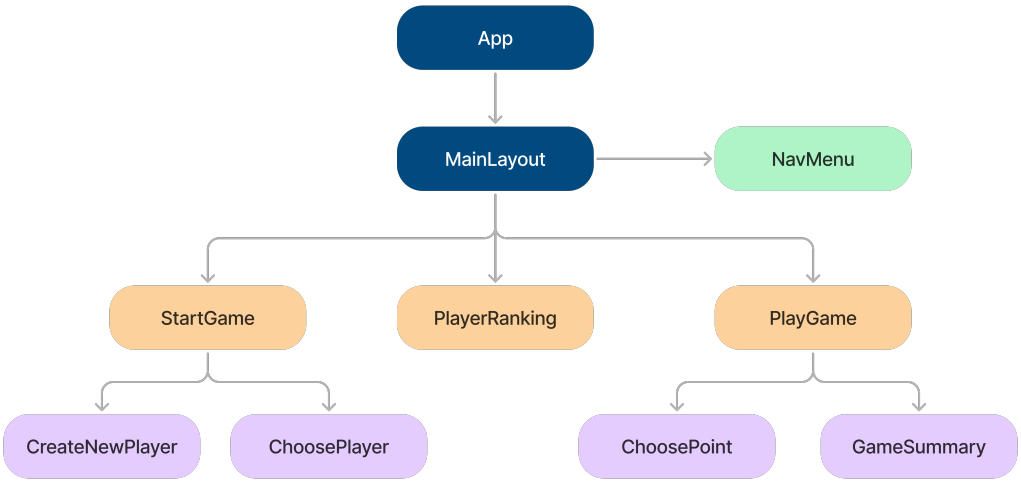

Figure 6.2: Component structure of applications

These components work together to create an interactive user experience. Below is an overview of the components and how they contribute to the DartPointTracker application:

- **App** – Serves as the root component of the DartPointTracker application, responsible for initializing the app and handling global settings.
- **MainLayout** – Defines the main layout and overall structure of the application, including the header, footer, and other common user interface elements shared across different pages. It serves as a container for other components and dictates how content is organized and presented throughout the app.
- **NavMenu** – Provides the navigation menu within the MainLayout, allowing users to easily switch between different views such as starting a new game, viewing player rankings or creating new players. It contains links that direct users to the main sections of the application.
- **StartGame** – This component serves as the initial page that is displayed when the application is opened. It allows users to configure a new game by selecting the target score and managing player participation. Users can select from predefined game modes (e.g., 301, 501), add existing players to the game by choosing from a list, or create new player profiles by entering names. The component includes modal dialogs for selecting players (**ChoosePlayer**) and creating new players (**CreateNewPlayer**). StartGame handles the setup phase by defining the game parameters and participant details necessary to initialize the game environment.
- **PlayGame** – This component manages active gameplay, allowing players to input their scores after each dart throw using the **ChoosePoint** component and displaying real-time scores for each player. Tracks game progress by managing turns, updating scores, and determining when the game ends. Upon completion of the game, the component presents a **GameSummary** with final scores and standings. Additionally, it provides the option to send game data to a web server to update players' Elo rankings, ensuring that standings are adjusted based on the latest game results.
- **PlayerRanking** – Provides a view of all players' Elo rankings, displaying their current standings based on previous game results.

6.1. PWA Features

To support the primary objective of evaluating the PWA development process, the following PWA features were implemented in the DartPointTracker application:

- **Installability**
  DartPointTracker supports installability by incorporating a Web App Manifest file. This file includes key metadata such as the application name, start URL, icons, and description. By providing this information, the browser can detect the app as a PWA and prompt users to install it on their devices. This enables users to launch the app directly from their home screen, offering an experience similar to that of a native application. The application runs in a standalone window, removing the standard browser UI elements to further enhance this native-like behavior.
- **Offline Functionality**
  The application provides extensive offline capabilities, allowing the core functions to be accessed without an Internet connection. Game logic and scorekeeping operations are handled locally on the device, allowing users to start new games and track scores without network access. Completed games are stored on the user's device and synchronized with the server when connectivity is restored, ensuring the data is up to date. Upon synchronization, player statistics and Elo rankings are updated automatically, and users are notified once the synchronization is complete.

  Although most of the core features are accessible offline, there are some limitations. Creating new players requires an internet connection to prevent duplicate names, and users are notified to go online for this action. Additionally, player rankings may not reflect the latest updates when offline, as cached data is displayed until the user reconnects, at which point the rankings are refreshed.
- **Responsive Design**
  DartPointTracker implements responsive design principles to ensure that the user interface functions effectively on various devices and screen sizes. The layout automatically adapts to different screen resolutions and orientations, providing a consistent experience on both mobile devices and desktop computers.
- **Secure Context**
  The application is served over HTTPS, ensuring a secure environment for users. This secure context is essential for enabling features such as service workers and offline functionality.

6.2. Responsive design

The design of the two applications is based on the Blazor Progressive Web Application template, which provides the foundation for its responsive layout. The page adapts to varying screen sizes, adjusting its structure accordingly. On larger screens (641px and above), the layout is divided into two main sections: a sidebar on the left and a main content area on the right. The sidebar, which displays a gradient background that transitions from dark blue to purple, remains fixed during scrolling and provides access to the navigation menu. The main content area takes up the remaining space and includes padding on both sides to maintain consistent spacing and alignment. At the top of the page, a navigation bar displays the application name "Dart Point Tracker" and stays visible as the user scrolls down.

On smaller screens (below 641px), the layout switches to a single-column design. The sidebar

collapses and can be accessed via a hamburger menu icon located in the top navigation bar. When the icon is clicked, the sidebar expands to cover the full width of the screen, providing navigation options. The top navigation bar remains fixed at the top, while the main content is displayed below and adjusts to occupy the full width of the screen. The layout is designed to rearrange itself based on screen size, ensuring that the sidebar, top bar, and content area are properly positioned on both large and small screens. The desktop and mobile layouts are illustrated in Figures 6.3 and 6.4 respectively.

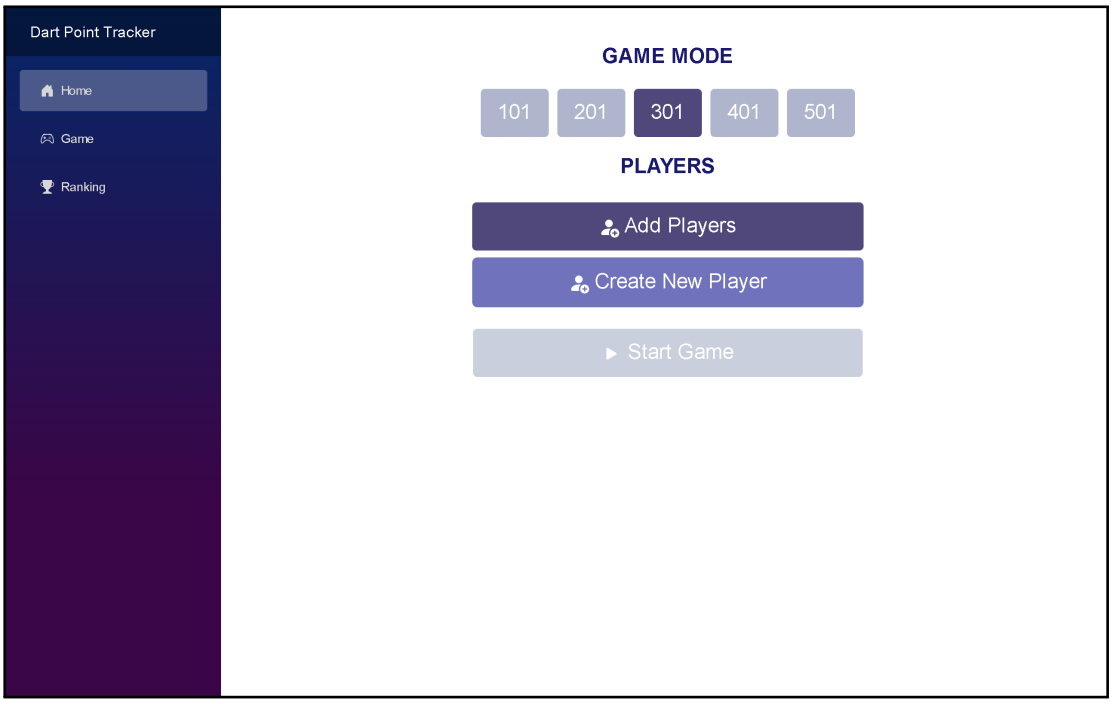

Figure 6.3: Desktop design

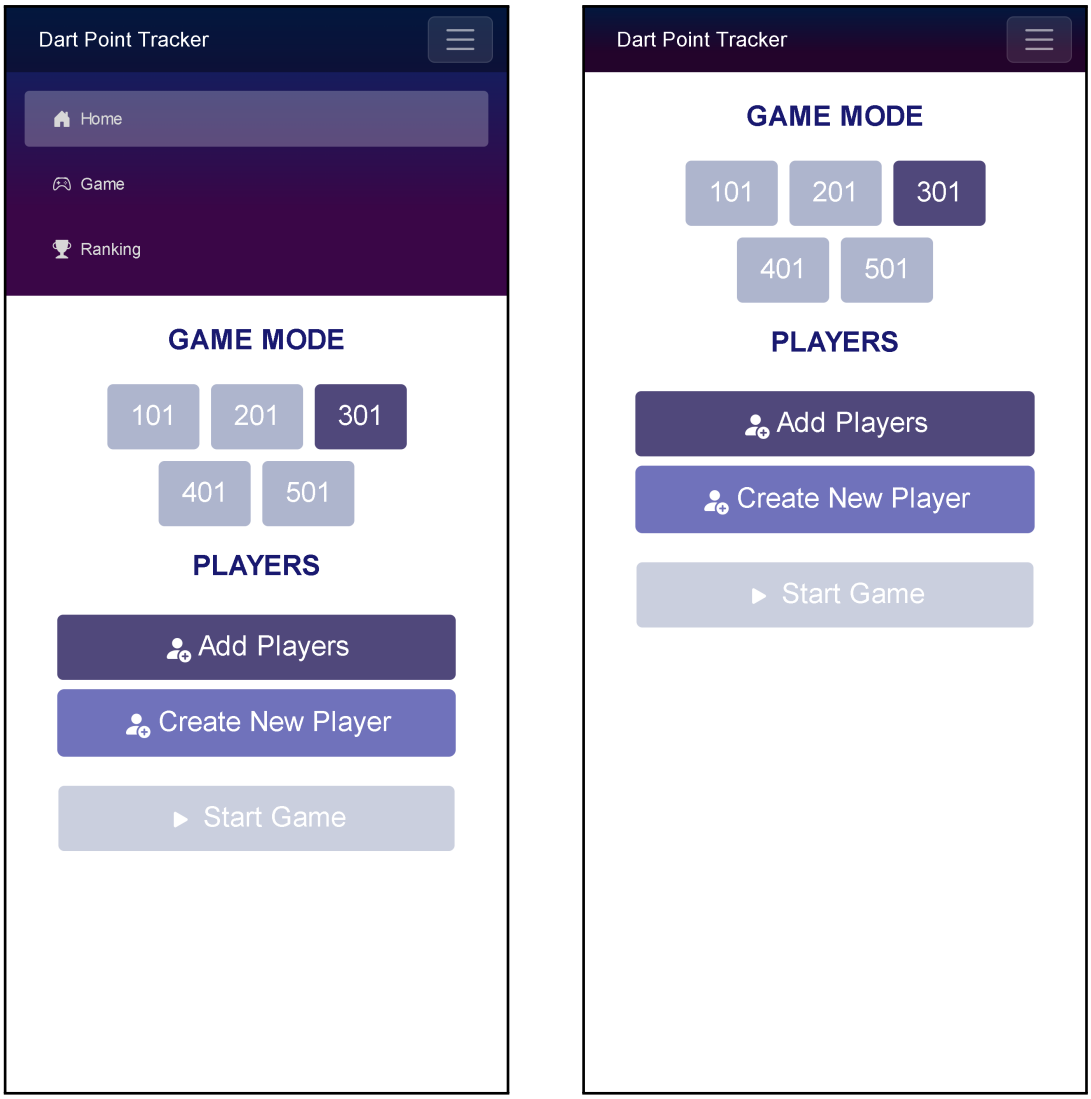

Figure 6.4: Mobile Design

6.3. Project Setup

In setting up a Blazor WebAssembly project, the process is closely integrated with the .NET ecosystem, requiring the installation of the **.NET Software Development Kit (SDK)**. Once the SDK is installed, a Blazor WebAssembly project with PWA support can be created using the .NET Command Line Interface (CLI) with the following command:

```
$ dotnet new blazorwasm --pwa -o DartPointTracker
```

The `-o` option in the command stands for *"output"*. Specifies the directory where the new project will be created and the project name. In this case, **DartPointTracker** is the name of the folder that will contain the generated project files. If the `-o` option is omitted, the project will be created in the current directory.

Alternatively, Visual Studio provides a graphical interface to create the project, where selecting the *"Progressive Web Application"* option during setup automatically includes the necessary PWA configuration. In both approaches, the project is scaffolded with essential PWA features, including the manifest file and service worker, pre-configured, ensuring that the PWA functionality is ready without requiring

further manual steps.

In contrast, setting up PWA functionality in a **Vue** project requires a bit more manual configuration. The first prerequisite is having **Node.js** version 18.3 or higher installed on the machine, as this version is required for **Vite** to be used as the development server and build tool. Once this requirement is met, the project initialization begins by executing the following command:

```
$ npm create vue@latest
```

This command prompts a series of questions to help configure the project based on the developer's preferences. Below is an example of the setup used to create the **DartPointTracker** application:

```
✓ Project name: ... DartPointTracker
✓ Package name: ... dartpointtracker
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add an End-to-End Testing Solution? » No
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes
✓ Add Vue DevTools 7 extension for debugging? (experimental) ... No / Yes
```

After providing the responses, the project is scaffolded with all the necessary files and configurations. The next step is to install the required dependencies to run the application by executing the following command:

```
$ npm install
```

To enable PWA functionality, the **vite-plugin-pwa** was chosen for the project. This plugin simplifies the integration of PWA features, such as service worker support and manifest configuration. It can be installed with the following command:

```
$ npm install vite-plugin-pwa
```

Once the plugin is installed, it must be configured in the project's `vite.config.js` file. This final step involves setting up the service worker and defining PWA-related settings, such as caching strategies, app icons, and the application manifest file. This manual configuration enables the full range of PWA features in the **DartPointTracker** application.

6.4. Manifest

Both applications have a manifest file with a .webmanifest extension. The only difference between them is in the method of configuring the manifest. In the Vue+Vite application, the configuration is carried out via the vite.config.js file, and the manifest is automatically generated. In Blazor, the file is configured directly. The resulting file is identical in both applications and is presented below.

Listing 6.1: Manifest of DartPointTracker app

```
{
    "name": "DartPointTracker",
    "short_name": "DartPointTracker",
    "start_url": "./",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#03173d",
    "description": "Dart Point Tracker is a Progressive Web
        ↪ Application designed to help users easily track and manage
        ↪ scores during darts games.",
    "icons": [{
            "src": "192.png",
            "sizes": "192x192",
            "type": "image/png",
            "purpose": "any"
        },
        {
            "src": "512.png",
            "sizes": "512x512",
            "type": "image/png",
            "purpose": "any"
        },
        {
            "src": "maskable_icon_x192.png",
            "sizes": "192x192",
            "type": "image/png",
            "purpose": "maskable"
        },
        {
            "src": "maskable_icon_x512.png",
            "sizes": "512x512",
            "type": "image/png",
            "purpose": "maskable"}],
    "screenshots": [{
            "src": "screenshot.png",
            "sizes": "1290x2796",
            "type": "image/png",
            "form_factor": "narrow",
            "label": "Home Page"
        },
        {
            "src": "screenshot_wide.png",
            "sizes": "2560x1600",
            "type": "image/png",
            "form_factor": "wide",
            "label": "Game Page"}]}
```

The manifest contains all the essential fields required for the browser to allow app installation. The "display" property is specifically set to "standalone," which enables the application to open and function similarly to a native standalone app. This setting ensures that the app launches in a separate window with its own icon in the app launcher, and browser UI elements such as the URL bar are excluded.

In addition to the standard 192×192 and 512×512 icons required for app installability, the manifest includes two icons with the "maskable" purpose. Maskable icons are used because different platforms have preferred icon shapes and web applications need to adapt across multiple platforms. By specifying the maskable purpose, the application allows user agents to apply platform-specific masks to the icons. This ensures that the icons integrate well with the aesthetics of various platforms.

The manifest also includes an array of screenshots and a description. Each screenshot features the "form_factor" attribute, which indicates to the browser whether the image is intended for desktop (wide) or mobile (narrow) environments. Including at least one screenshot for each form factor is essential to enable the Richer Install UI dialog, an experimental interface designed to help users recognize the app they are adding to their device. This dialog is not available in all browsers; it is currently implemented in Chrome, where the installation dialog appears on mobile platform as shown in Figure 6.5.

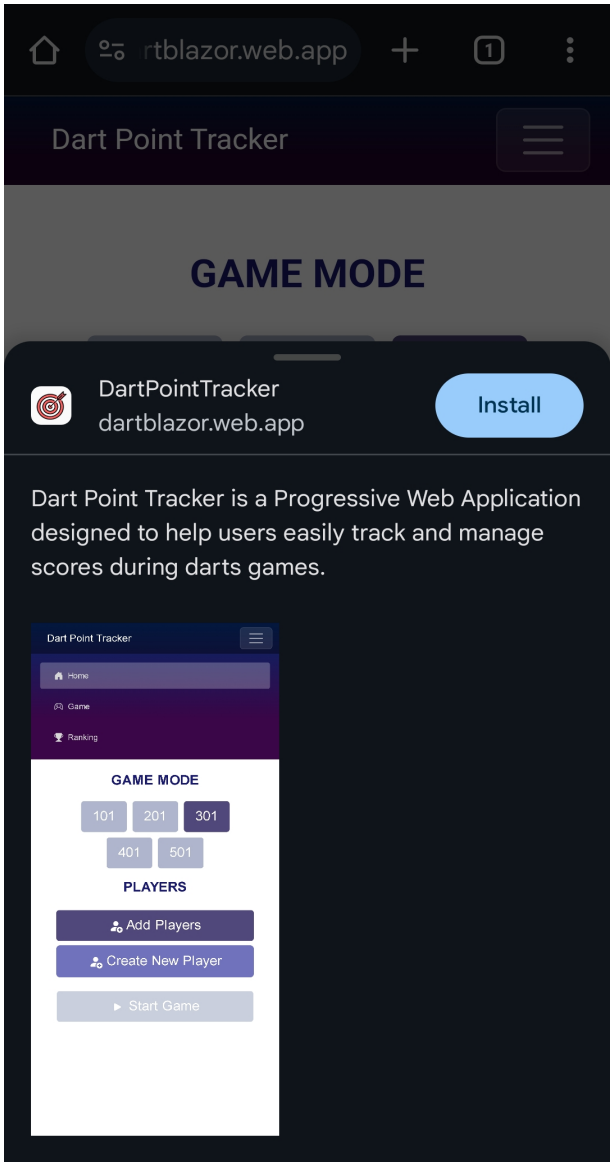

Figure 6.5: Install dialog on mobile platform

6.5. Service Worker

6.5.1. Workbox integration

For the development of service workers, the Workbox library was used to simplify the implementation of caching strategies, background sync, and offline capabilities.

The Vue version of DartPointTracker benefits from a PWA plugin that is pre-configured with Workbox. This plugin automatically handles the integration of Workbox, allowing the service worker to utilize Workbox modules without requiring additional setup for module bundling. Since the Vue project employs a build tool with built-in bundling capabilities, Workbox functionality, including module imports, is directly incorporated during the build process.

In contrast, Blazor, which is primarily based on C# and .NET, requires additional setup to integrate JavaScript modules, as it does not include a bundler by default. To achieve similar functionality, there are two options:

1. Bundle Workbox Modules: Add a bundler to the project to include Workbox modules directly in the service worker. This approach involves additional setup and configuration, increasing the complexity of the build process.
2. Load Workbox from a Content Delivery Network (CDN): Import Workbox scripts directly from a CDN using `importScripts`.

In DartPointTracker's Blazor implementation, the CDN approach was chosen to streamline development and avoid the complexities of setting up a bundler within the Blazor project structure.

In both the Vue and Blazor applications, the service worker is registered at the root level (/). Both applications utilize Workbox's **injectManifest** strategy that enables custom service worker implementations.

For the Vue application, the vite-plugin-pwa configuration(Listing 6.2) specifies the source directory, the service worker filename, and defines the extensions of files to be precached. The configuration ensures that essential assets such as HTML, web manifest, JSON, JavaScript, CSS, SVG, PNG, and ICO files are cached for offline use and efficient loading.

Listing 6.2: PWA Plugin Configuration for Vue Application

```
VitePWA({
  strategies: 'injectManifest',
  srcDir: 'src',
  filename: 'sw.js',
  workbox: {
    globPatterns: ['**/*.{html,webmanifest,json,js,css,svg,png,ico}']
  }
})
```

In the Blazor application, a configuration file(Listing6.3) was created to manage the service worker settings. This configuration includes all the file types defined for precaching in the Vue application, with the addition of `.wasm` and `.dll` files, which are essential for Blazor WebAssembly's offline functionality. The configuration specifies the `globDirectory`, indicating the directory where Workbox will scan for files to precache; this directory corresponds to where the production build files are located.

Listing 6.3: Workbox Configuration for Blazor Application

```
module.exports = {
  globDirectory: "release/wwwroot/",
  globPatterns: [
    '**/*.{html,webmanifest,json,js,css,svg,png,ico,wasm,dll}'
  ],
  swSrc: 'sw.js',
  swDest: 'release/wwwroot/sw.js',
};
```

After completion of the production build of the Blazor application, the Workbox CLI is used to inject the precache manifest into the service worker in the specified destination with the following command:

```
$ workbox injectManifest workbox-config.js
```

It is important to note that for precaching to function correctly with the injectManifest strategy, the precacheAndRoute function must be used within the custom service worker.

6.5.2. Service worker implementation

Both applications employ the same functionalities in their service workers; however, the implementation differs slightly. Since the Blazor application uses a CDN to load the Workbox library, it requires specifying the full path for each module to use the library's features. In contrast, the Vue application, which bundles all imports into a single file during the build process, does not require specifying module paths individually.

The custom service worker code in both applications can be divided into four main parts: updating the service worker, precaching assets, handling background synchronization, and runtime caching of API calls.

Updating

Listing 6.4: Updating Service Worker

```
self.skipWaiting();
clientsClaim();
```

The code in Listing6.4 ensures that updates to the service worker take effect immediately. The self.skipWaiting() function forces the newly installed service worker to become active immediately, bypassing the default waiting period. While this makes the new service worker active, it does not automatically mean that currently loaded clients (open pages) are controlled by it. Instead, it ensures that any new clients will use this version of the service worker moving forward.

To address currently open clients, the clientsClaim() function is then called. This ensures that the active service worker immediately takes control of all existing pages within its scope, without requiring users to manually reload those pages.

Precaching

Listing 6.5: Precaching code

```
workbox.precaching.cleanupOutdatedCaches();
workbox.precaching.precacheAndRoute(self.__WB_MANIFEST);
```

The code in Listing6.5 demonstrates the invocation of two functions from the `workbox.precaching` module. The **cleanupOutdatedCaches()** function adds an event listener to the service worker's `activate` event, which is responsible for cleaning up old, incompatible caches created by previous versions of Workbox. This ensures that outdated cache data is removed, freeing up storage space, and preventing conflicts with the updated version of the service worker.

The **precacheAndRoute(self.__WB_MANIFEST)** function precaches assets listed in the precache manifest, which replaces the placeholder variable `self.__WB_MANIFEST` in the service worker. It performs two key tasks:

- **Precaching**: It adds the specified assets to the precache list, removing any duplicates, and stores the files in the cache during the service worker's installation phase.
- **Routing**: It automatically registers a fetch event listener that responds to network requests with the precached assets. If a request matches a file in the precache, the service worker serves the cached version. For requests that are not in the precache, the fetch event falls through to other fetch event listeners or network requests, allowing the app to handle uncached resources.[34]

Runtime Caching of API Calls

Listing 6.6: Runtime caching for getting players

```

    workbox.routing.registerRoute(
        ({ url }) => url.pathname.startsWith('/Players'),
        new workbox.strategies.NetworkFirst({
            cacheName: 'players-api-cache',
            networkTimeoutSeconds: 10,
            cacheableResponse: { statuses: [200] },
        }),
        'GET'
    );
```

The code in the Listing6.6 above registers a route for GET requests to URLs that begin with /Players. This route uses Workbox's NetworkFirst strategy, which prioritizes fetching fresh player data from the network whenever the user is online. The service worker attempts to retrieve the latest player data by sending a request to the server. If the request is successful (within a 10-second network timeout), the response is cached in the players-api-cache for future use.

If the user is offline or if the network request fails (e.g., due to network latency or timeouts), the service worker falls back to serving the cached player data. This cached data corresponds to the most recent successful request made while the user was online. In this way, the user can continue to interact with the player data previously fetched, even in offline scenarios.

Background Sync

Listing 6.7: Background Sync Implementation

```
const bgSyncPlugin = new BackgroundSyncPlugin('gameQueue', {
  maxRetentionTime: 24 * 60
});

registerRoute(
  ({ request }) => request.url.includes('/Game') && request.method === 'POST',
  new NetworkOnly({
    plugins: [bgSyncPlugin]
  })
);

self.addEventListener('sync', (event) => {
  if (event.tag === 'gameQueue') {
    event.waitUntil(
      self.registration.showNotification('Game save completed!', {
        body: 'Your offline game has been successfully saved!',
        icon: '/favicon.svg'
      })
    );
  }
});
```

The code in the Listing6.7 above establishes a background synchronization mechanism to send game data to the web server. A BackgroundSyncPlugin instance named bgSyncPlugin is created with the queue identifier 'gameQueue' and a maxRetentionTime of 24 hours, which defines the period during which failed requests will be retried.

The service worker then registers a route specifically targeting POST requests to URLs containing /Game, which handles requests related to sending game data. The NetworkOnly strategy is applied, meaning the data is sent directly to the network without using cached responses. If the network is unavailable and the request fails, bgSyncPlugin queues the failed submission and automatically retries once the connection is restored.

The service worker also listens for **sync events** via the self.addEventListener('sync', ...) method. When a sync event with the tag 'gameQueue' is triggered, indicating that the background sync process is attempting to resend queued requests, the event listener is activated. The event.waitUntil() method is used to delay the execution of any actions until the background sync process completes. Inside this callback, a notification is displayed using the self.registration.showNotification() method, which informs the user that the game data has been successfully saved after being synchronized. The notification includes a custom message and an icon.

In order for the user to receive notifications, their permission is required. This permission is requested on the first page of the application using the JavaScript code presented in a Listing6.8.

Listing 6.8: Function for Requesting Notification Permission

```
window.requestNotificationPermission = () => {
    if ('Notification' in window) {
        if (Notification.permission === 'default') {
            Notification.requestPermission().then(permission => {
                if (permission === 'granted') {
                    console.log('Notifications enabled.');
                } else if (permission === 'denied') {
                    console.log('Notifications denied.');
                }
            });
        }
    } else {
        console.log('This browser does not support notifications.');
    }
};
```

The provided function checks if the browser supports the Notification API, and if the user has not already granted or denied permission, it prompts them to allow or block notifications using Notification.requestPermission(). In the Blazor application, we use the IJSRuntime service to invoke this function. IJSRuntime is an interface that enables JavaScript interop, allowing C# code to call JavaScript functions asynchronously. Since there is no WebAssembly API to request notification permissions, Blazor relies on this interop mechanism to request them. In Vue, this function can be called directly from the JavaScript context.

6.6. Hosting

To evaluate and compare the implementation of Progressive Web Applications using Blazor and Vue.js, both versions of the DartPointTracker application were deployed in a live environment. This deployment is necessary to test key aspects, such as performance, user experience, and offline functionality under real-world conditions, ensuring accurate analysis of application behavior when accessed over the Internet.

The Firebase platform was selected as the hosting solution for both applications.Firebase provides a simple deployment process via the CLI interface, so updates can be quickly deployed with a single command (firebase deploy). It also offers automatic Secure Sockets Layer (SSL) certificates, which are necessary for the functionality of the service worker, ensuring that both applications meet HTTPS requirements. [35]

Hosting both applications on the same platform ensures consistency in the testing environment. Any observed differences in performance, behavior, or user experience can be attributed to the frameworks, rather than to differences in hosting infrastructure or server configuration.

7. COMPARISON OF PWA IMPLEMENTATIONS

The objective of this study is to explore the feasibility of using Blazor technology to implement Progressive Web Applications. In the previous chapter, the feasibility of implementing the core PWA features with Blazor was demonstrated, including service worker integration, asset precaching, offline functionality, installability, and background synchronization. The DartPointTracker application successfully incorporated these features, confirming that Blazor meets the technical requirements for PWA development.

This chapter focuses on the second research question: What are the advantages and disadvantages of using Blazor to build PWAs in comparison to JavaScript-based technologies? To answer this, we will compare the performance and developer experience of building the DartPointTracker application in Blazor and Vue.js. The performance comparison will cover key metrics such as First Contentful Paint, Largest Contentful Paint, Total Blocking Time, and Speed Index for the first-time and repeat-visit experience. Furthermore, differences in storage usage for cached assets will be examined, along with the rendering performance of each application when processing requests from the API.

The developer experience will be evaluated by comparing the complexity and volume of code, measured in lines of code, and a comparison of development server startup times. This analysis will provide insights into how Blazor compares to Vue.js in terms of both performance and ease of implementation, offering a comprehensive understanding of Blazor's suitability for PWA development relative to JavaScript-based technologies.

7.1. Lighthouse performance test

This performance evaluation used version 12.2.0 of Google Lighthouse to measure key metrics related to the initial loading of two Progressive Web Applications. Lighthouse is an open source automated tool integrated with Chrome DevTools that offers in-depth insights into the performance, accessibility, and overall quality of web pages. This analysis focuses on metrics specifically relevant to the initial load performance of the applications, including:

- **First Contentful Paint (FCP)** - measures the time from when a user initially navigates to a page until any portion of the page's content appears on the screen. In this context, "content" includes text, images, SVG elements, or non-white canvas elements.[36]
- **Largest Contentful Paint (LCP)** - measures the time it takes for the largest visible content element to be fully rendered. Indicates how quickly the main content of the page is displayed.[37]
- **Total Blocking Time (TBT)** - measures the total amount of time after First Contentful Paint where the main thread was blocked and unable to respond to user interactions, such as scrolling or clicking.[38]
- **Speed Index** - evaluates how quickly content is visually displayed during page load. Lighthouse records a video of the page loading in the browser and calculates the visual progression between frames.[39]

Two sets of tests were conducted using Lighthouse under different conditions to evaluate the initial load performance of both applications:

1. **First-Time Visit (Clear Storage On):**
   In this scenario, Lighthouse was configured with the "Clear Storage" option enabled, which clears all cached resources and data associated with the web page before each test. This forces the application to download all necessary assets, simulating the experience of a first-time visitor.
2. **Repeated Visit (Clear Storage Off):**
   In this scenario, the "Clear Storage" option was disabled, allowing cached resources from previous visits to be used. This simulates a repeated visit where the browser can access locally stored data, measuring how well the application utilizes caching to speed up the loading process.

The tests were conducted using both device configurations available in Lighthouse: mobile and desktop. The testing conditions were as follows:

- **Mobile Environment:**
  Lighthouse emulates a mobile device (Moto G Power) with a 4x CPU slowdown and slow 4G network throttling (150 ms TCP round-trip time, 1,638.4 kb/s throughput). The screen resolution is set to 412x823 with a device pixel ratio of 1.75. This setup evaluates how well the applications perform under constrained mobile resources and slow network conditions.
- **Desktop Environment:**
  For desktop testing, Lighthouse simulates a more powerful desktop environment with a 1x CPU slowdown and custom network throttling (40 ms TCP round-trip time and 10,240 kb/s throughput). The screen resolution is also set to 412x823 with a device pixel ratio of 1.75.

Each test was run three times and the average result was calculated to ensure accurate and reliable performance measurements. The results are shown in Tables 7.1, 7.2, 7.3 and 7.4 for the first-time visit scenario, and in Tables 7.5,7.6,7.7 and 7.8 for the repeated visit scenario.

First-Time Visit (Clear Storage On)

	Attempt 1	**Attempt 2**	**Attempt 3**	**Average**
First Contentful Paint [ms]	342	335	337	338
Largest Contentful Paint [ms]	2142	2335	2303	2260
Total Blocking Time [ms]	866	880	912	886
Speed Index [ms]	1195	1203	1222	1206

Table 7.1: Blazor results on desktop device(Clear Storage enabled)

	Attempt 1	**Attempt 2**	**Attempt 3**	**Average**
First Contentful Paint [ms]	1273	1256	1263	1264
Largest Contentful Paint [ms]	12 598	13 181	13 188	12 989
Total Blocking Time [ms]	4657	4074	4086	4272
Speed Index [ms]	3301	2659	2808	2922

Table 7.2: Blazor results on mobile device(Clear Storage enabled)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	455	364	357	392
Largest Contentful Paint [ms]	455	378	377	403
Total Blocking Time [ms]	0	0	0	0
Speed Index [ms]	1046	364	357	589

Table 7.3: Vue results on desktop device(Clear Storage enabled)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	1394	1506	1391	1430
Largest Contentful Paint [ms]	1531	1582	1527	1546
Total Blocking Time [ms]	38	31	37	35
Speed Index [ms]	1394	1506	1391	1430

Table 7.4: Vue results on mobile device(Clear Storage enabled)

Repeated Visit (Clear Storage Off)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	63	44	64	57
Largest Contentful Paint [ms]	1373	1275	1309	1319
Total Blocking Time [ms]	927	873	857	885
Speed Index [ms]	1026	692	958	892

Table 7.5: Blazor results on desktop device(Clear Storage disabled)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	124	154	165	147
Largest Contentful Paint [ms]	4719	5111	4678	4836
Total Blocking Time [ms]	4051	4785	4030	4288
Speed Index [ms]	2442	2689	2483	2538

Table 7.6: Blazor results on mobile device(Clear Storage disabled)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	43	38	58	46
Largest Contentful Paint [ms]	65	52	58	58
Total Blocking Time [ms]	0	0	0	0
Speed Index [ms]	145	105	84	111

Table 7.7: Vue results on desktop device(Clear Storage disabled)

	Attempt 1	Attempt 2	Attempt 3	Average
First Contentful Paint [ms]	291	226	250	255
Largest Contentful Paint [ms]	291	226	250	255
Total Blocking Time [ms]	50	58	78	62
Speed Index [ms]	321	242	269	277

Table 7.8: Vue results on mobile device(Clear Storage disabled)

The performance evaluation using Google Lighthouse highlights significant differences between Blazor WebAssembly and Vue.js PWAs concerning initial load metrics. The data reveal that Blazor consistently underperforms compared to Vue on key performance indicators, particularly in the LCP and TBT, especially in mobile conditions.

Blazor WebAssembly applications require the download and initialization of the .NET runtime, application binaries, and dependencies before rendering any meaningful content. This initial overhead contributes to longer load times and increased TBT, as the main thread remains occupied with processing these resources. For first-time visitors with clear storage, Blazor recorded an average LCP of approximately 12,989 ms and a TBT of 4,272 ms on mobile devices. In comparison, Vue achieved an LCP of 1,546 ms and a TBT of 35 ms under the same conditions. The significant performance gap is more noticeable on mobile devices, where limited computational power and slower network speeds amplify the latency involved in downloading and initializing Blazor's larger payload.

Asset caching significantly improves nearly all performance metrics for both applications by reducing the need to download resources on repeated visits. In the case of Blazor, the LCP time decreases with caching because essential assets like the .NET runtime and application binaries are loaded from the local cache rather than fetched over the network. This reduction in network latency allows the main content to render more quickly, thus improving the LCP metric.

However, despite the improvement in LCP the TBT values are nearly identical on both device types, whether caching is enabled or not. This persistent high TBT is due to the computational overhead associated with Blazor's runtime initialization, which is not mitigated by caching. The main thread remains heavily occupied with parsing, compiling, and executing the WebAssembly modules and initializing the .NET runtime environment. These CPU-intensive tasks block the main thread for extended periods, preventing it from responding to user interactions promptly and contributing to the high TBT metrics.

Vue.js applications outperform Blazor in terms of Largest Contentful Paint (LCP), Total Blocking Time (TBT), and Speed Index. This is primarily due to Vue's lightweight architecture and its use of the browser's native JavaScript engine, eliminating the need for additional runtimes. As a result, Vue.js applications benefit from quicker parsing and execution of scripts, leading to lower LCP and TBT values. For instance, with caching enabled on mobile devices, Vue.js achieved an LCP of 255 ms and a TBT of 62 ms, significantly outperforming Blazor in these metrics. Additionally, Vue.js exhibits lower Speed Index values, reflecting its efficiency in rendering content rapidly and providing a smoother visual progression during page load.

Although Blazor shows slightly better results in First Contentful Paint (FCP), this advantage is largely attributed to the immediate display of a loading animation embedded directly in the index.html file, which is registered as the FCP. However, this early visual feedback does not translate to faster rendering of the main content.

7.2. Application Size

PWAs enhance user experience through caching essential application data. This caching mechanism stores resources locally on the user's device, enabling the application to function without a network connection and reducing the need to re-download assets on subsequent visits. To compare the storage requirements of Blazor WebAssembly and Vue.js PWAs, we measured the sizes of their cached assets using Chrome DevTools and the Cache API.

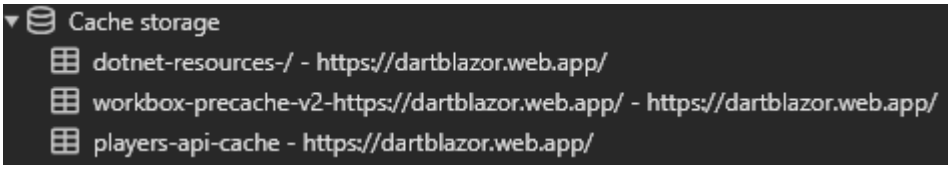

Figure 7.1: Cache Storage tab of Blazor PWA

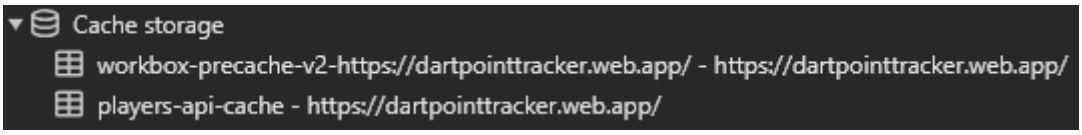

Figure 7.2: Cache Storage tab of Vue PWA

Figures 7.1 and 7.2 show the Cache Storage tab in Chrome DevTools for both applications. Both use caches to store player data and pre-cached assets managed by their service workers, as detailed in the development chapter. The Blazor application includes an additional dotnet-resources cache, specific to Blazor WebAssembly, that contains the .NET runtime and application binaries essential for execution within the browser. The cache sizes are presented in Table 7.9:

Application	Cache Name	Size (KB)
Blazor PWA	`dotnet-resources`	6,705.28
	`workbox-precache-v2`	3,893.07
Vue PWA	`workbox-precache-v2`	530.37

Table 7.9: Cache Sizes for Blazor and Vue PWAs

The player data cache was not measured in either application, as it does not contain application files but rather dynamic data fetched from APIs.

The difference in cache sizes between Blazor and Vue.js applications is mainly due to their architectural designs. Blazor WebAssembly requires the .NET runtime and associated libraries, adding about 6.7 MB to its cache, along with additional pre-cached assets, bringing the total to over 10 MB. Its `workbox-precache-v2` cache includes not just static assets but also larger compiled WebAssembly files and DLLs, which contain application logic.

In contrast, Vue.js applications are smaller because they utilize the browser's native JavaScript engine without needing large runtimes or binary files. The `workbox-precache-v2` cache for Vue.js mainly includes minified JavaScript, CSS, and optimized images. Vue.js benefits from build optimizations like tree-shaking and code splitting, reducing the size of cached assets to around 530.37 KB. The minimalistic design and fewer bundled dependencies result in a smaller cache footprint.

7.3. Rendering performance

An analysis was conducted to evaluate the rendering performance of Blazor and Vue by measuring the time required for UI elements to appear on the screen. The focus was on the ranking page, where players are displayed in descending order based on their Elo scores. When navigating to the ranking page, an API request is made to retrieve player data. While waiting for the response, a loading screen is displayed. Once the response is received, the data is deserialized, the players are sorted by their Elo scores, and ranks are assigned before the table is rendered.

Performance measurements were performed using the `Stopwatch` class in Blazor and the `performance` API in Vue, which allow for detailed measurement of the elapsed time. The measurements included four main tasks:

- **API Request Handling:** The time difference between the total execution time of the method that fetches players and the actual API request duration, as recorded in the Chrome DevTools Performance panel.
- **Sorting and Rank Assignment:** The time required to sort players based on their Elo scores and assign ranks accordingly.
- **UI Diffing:**
  - In **Blazor**, UI diffing is measured from the moment `IsLoading = false` is set until `OnAfterRenderAsync` fires, capturing the complete cycle of state change detection. During this period, Blazor reevaluates the component's render tree in WebAssembly, compares it with the previous state to identify differences, and generates the necessary DOM operation instructions. The measured time represents how long it takes Blazor to perform diffing calculations in WebAssembly and communicate the required DOM updates across the WebAssembly-JavaScript boundary.
  - In **Vue**, UI diffing is measured from the moment `isLoading.value = false` is set until the `nextTick` callback executes, capturing Vue's reactive update cycle. During this time, Vue detects the state change, re-renders the Virtual DOM tree, and compares it with the previous tree to determine the minimal set of DOM operations needed.
- **Rendering:** Measured using the Chrome DevTools Performance panel, capturing only the browser's rendering tasks, including style recalculation, layout computation, pre-paint, paint, and commit operation. This measurement is separate from framework-level diffing and represents the browser's work in applying the calculated UI changes.

Measurements were conducted with different numbers of players, ranging from 100 to 5,000. For each dataset size, ten measurements were recorded, and the mean values were calculated. The results are presented in Table 7.10.

Number of Players	Blazor				Vue			
	Request	**Sort**	**Diffing**	**Render**	**Request**	**Sort**	**Diffing**	**Render**
100	14.6	0.6	60.3	15.5	0.4	0.6	2.0	15.7
500	23.2	1.6	290.5	64.6	0.9	2.9	6.5	58.7
1000	62.4	4.4	534.2	141.9	1.2	4.6	15.6	115.6
2000	102.5	8.9	1152.5	261.2	2.2	11.3	31.6	225.9
5000	186.7	18.4	2805.1	642.5	4.7	32.0	73.8	550.6

Table 7.10: Detailed execution times (in milliseconds) for Blazor and Vue

Based on the results we can observe that Vue consistently outperforms Blazor in API request handling, with Vue processing requests up to 40 times faster. When making HTTP requests from WebAssembly, the calls must traverse the WebAssembly-JavaScript boundary, since network access is controlled by the browser's JavaScript environment. Blazor's HTTP client implementation adds additional abstraction layers over browser fetch APIs that introduce performance overhead compared to Vue's direct JavaScript approach.

Sort and rank assignment times remain relatively similar for both frameworks with smaller data sets, but Vue starts to show slower performance as the number of players increases. For 5,000 players, Vue's sort time is actually higher than Blazor's, indicating WebAssembly's advantage in computational tasks that do not involve DOM manipulation or framework-specific operations.

The most significant performance difference is observed in the UI diffing phase, where Vue completes the process significantly faster than Blazor. Vue's diffing algorithm processes changes 30 times faster for 100 players, and this difference increases to 38 times faster for 5000 players, where the UI diffing phase takes nearly three seconds in Blazor. Both frameworks perform UI diffing by comparing the new UI representation with the previous state, but their approaches differ fundamentally. Blazor constructs a Render Tree and executes its diffing algorithm within the .NET runtime running on WebAssembly, which then requires communication of DOM updates across the WebAssembly-JavaScript boundary. In contrast, Vue employs a Virtual DOM diffing approach that operates entirely within JavaScript, benefiting from modern JavaScript engine optimizations and avoiding the additional overhead caused by boundary-crossing operations. This difference in execution environment contributes to the performance gap observed between the two frameworks.

Both frameworks appear to compute the minimal set of UI changes with similar accuracy, as evidenced by the comparable rendering times observed in the browser rendering phase. The data show that rendering times remain relatively consistent between Vue and Blazor across all dataset sizes, indicating that both frameworks are generating similar sets of DOM operations for the browser to process.

7.4. Volume of code

When developing web applications, the volume of code can provide insights into the complexity and maintainability of an implementation. To evaluate the differences between Blazor and Vue in the development of PWAs, a comprehensive analysis was performed across different layers of the application: the presentation layer, the infrastructure layer, the domain layer and the startup code. The code was consistently formatted for a fair comparison, excluding blank lines, comments, and lines containing only brackets to focus solely on meaningful code.

7.4.1. Presentation Layer

To compare the conciseness of the Blazor and Vue frameworks in the implementation of PWAs, a detailed measurement of Lines of Code (LOC) across various components was conducted. It was divided into two sections: the HTML part (Razor syntax in Blazor and templates in Vue) and the code part, which includes the logic of each component.

	Blazor			**Vue**		
	html	**code**	**sum**	**html**	**code**	**sum**
MainLayout	10	1	11	12	1	13
NavMenu	44	4	48	46	4	50
StartGame	69	40	109	71	47	118
PlayGame	51	19	70	52	23	75
PlayerRanking	27	15	42	29	19	48
ChoosePlayer	29	35	64	29	43	72
CreateNewPlayer	27	27	54	27	26	53
ChoosePoint	8	22	30	10	23	33
GameSummary	22	2	24	23	6	29
Router	0	0	0	0	11	11

Table 7.11: Lines of code comparison between Blazor and Vue PWA

The HTML sections in both frameworks show remarkable similarity in LOC, with differences typically ranging between one and two lines. These small differences are due to the following.

- **Template Markup in Vue**: Vue's `<template>` tags add two extra lines.
- **Loops in Blazor**: Blazor's loops add an extra line compared to Vue's `v-for`, which is inside the HTML tag.
- **Modal Implementation**: In components like `ChoosePlayer` and `CreateNewPlayer`, Blazor's modal implementation adds two extra lines, making the total LOC the same in both frameworks for these components.

In the code sections, Vue generally has more LOC in most components due to several factors:

- **Import Statements**: Vue components often require explicit import statements for necessary modules and components. Each module or component is typically imported on a separate line, increasing the overall LOC.
- **Props Definition vs. Parameters**: Vue is more verbose in defining props, requiring explicit declarations and type definitions, which adds extra lines compared to Blazor's more concise parameter definitions.

- **Modal Implementation**: Vue requires additional code in the parent component to manage modals, such as writing functions to control them. In contrast, Blazor handles more of the modal logic within the modal component itself.
- **Routing**: In Vue, routing requires explicit configuration, which results in an additional 11 LOC in the **router.ts** file. Each route must be manually defined and registered, specifying the path and the corresponding component. In Blazor, routing is automatically managed through the use of the **@page** directive in components.

7.4.2. Infrastructure Layer

A subsequent LOC measurement was conducted on the `PlayerService` file, located within the application infrastructure layer. This file contains three methods, each responsible for handling a different type of API request:

- **CreatePlayer**: POST request with a query string parameter.
- **SendGame**: POST request with a JSON-formatted request body.
- **GetPlayers**: GET request and retrieves an array of Player objects in JSON format.

The results of this analysis are presented in Table 7.12:

Method	Blazor	Vue
CreatePlayer	12	13
SendGame	13	15
GetPlayers	9	14
Other	5	9

Table 7.12: Comparison of LOC in Player Service Methods

The results indicate that Vue's infrastructure code requires more lines of code, with this amount varying between methods. The primary factor influencing the line count differences between Blazor and Vue implementations is Blazor's use of dependency injection for `HttpClient` configuration. It eliminates the need to repeatedly specify the API URL and authorization headers across all service methods. This pre-configured `HttpClient` is simply injected into the `PlayerService` constructor, making it available throughout the service. In contrast, the Vue implementation requires manual specification of the API URL and authorization headers for every request, adding repetitive configuration code for all methods.

More configuration is needed in the `SendGame` method in Vue. A method must be explicitly set to define the request as `POST` with a header that specifies the request body as `application/json`. Furthermore, JSON serialization must be handled manually using `JSON.stringify()`. In contrast, Blazor's `PostAsJsonAsync` automatically applies the necessary headers and serialization, reducing the verbosity.

The most significant difference in LOC is observed in the `GetPlayers` method. Vue requires error checking to confirm a successful request, followed by manual parsing with `await response.json()` and mapping each object to a `Player` instance. In Blazor, `GetFromJsonAsync` simplifies this process by fetching data and automatically deserializing the JSON response into a list of `Player` objects in a single line. Any unsuccessful request triggers an `HttpRequestException`, which is handled within a `try-catch` block, eliminating the need for additional response validation.

7.4.3. Domain Layer

A subsequent analysis was conducted to measure the volume of the domain-specific code. In addition to the LOC used in the presentation and infrastructure layers, this analysis included measurements of characters (excluding whitespace characters) and characters per line. These additional metrics are valid in the context of the domain, as the code is completely independent and free from dependencies on external services such as httpClient or fetch API. The evaluation focused on the core domain classes, Player, Game and GameService, which contain the core business logic of the application. These classes have been implemented in a consistent manner and all properties and methods have been uniformly named, ensuring that any differences observed can only be attributed to language features of the respective implementations. The results of the analysis are presented in Table 7.13:

	Blazor			**Vue**		
	LOC	**Ch**	**Ch/L**	**LOC**	**Ch**	**Ch/L**
Player	48	1525	31.77	61	1964	32.2
Game	48	1508	31.42	58	1731	29.84
GameService	6	234	39	9	337	37.44

Table 7.13: Comparison of LOC, Characters, and Characters per line

The analysis demonstrates clear differences in the volume of code between the Blazor (C#) and Vue (TypeScript) implementations. The Blazor version requires fewer lines of code, and the number of characters per line remains comparable in both implementations. This indicates that the code volume is significantly higher in the TypeScript implementation. The primary factors contributing to this outcome are as follows:

- **Primary Constructor:** This is a syntax introduced in C# 12 that allows a constructor to be defined directly in the declaration of type and whose parameters are available anywhere in the body of type.[40] This allows the initialization of a property to be combined with its declaration. In contrast, in TypeScript, properties must be declared before the constructor, and the constructor method must explicitly assign each parameter to the corresponding property, resulting in numerous assignment statements that increase code length.

- **"this" Keyword:** TypeScript requires explicit "this" references for every property and method access within class methods. This requirement adds extra characters to nearly every line of operational code within class methods. In C#, the "this" keyword is optional in most contexts, as class members can be accessed directly by name without self-reference.

- **Collection Operations:** C# makes use of LINQ expressions, which provide a declarative SQL-like syntax for collection operations. These expressions are often more readable and concise than TypeScript's approach using array methods like `filter`, `map`, and `reduce`.

- **Collection Indexing:** C# simplifies accessing elements from the end of a collection with the caret (`^`) operator, while TypeScript uses the collection's length, resulting in longer expressions.

- **Computed Properties:** C# simplifies computed properties with expression-bodied members (`=>`), avoiding the function blocks, braces, and return statements needed in TypeScript getters.

7.4.4. Application Startup

The last measurement of the volume of the code was carried out on the application startup code to compare the initialization process of Blazor and Vue. The results are in Table 7.14

File	LOC
Program.cs (Blazor)	13
main.ts (Vue)	7

Table 7.14: LOC in Application Startup Files

Program.cs in Blazor has more LOC than main.ts in Vue, where the difference is that they have different responsibilities. The Blazor file initializes the application by configuring the host builder, setting up dependency injection, and defining the HTTP client with api url and basic authentication. It also specifies the root components and then runs the Blazor WebAssembly application. Meanwhile, the Vue file focuses on importing Bootstrap, configuring the router and store, and creating the Vue application instance. It then mounts the application to the DOM by targeting the #app element.

7.4.5. Overall Code Volume Analysis

Table 7.15 presents the overall LOC results for the previously measured application layers, along with the percentage difference, to illustrate the variations of the code volume between Blazor and Vue in the development of a PWA.

	Blazor	Vue	Difference (%)
Presentation - HTML	287	299	+4.18%
Presentation - Code	165	206	+23.03%
Infrastructure	39	51	+30.77%
Domain	102	128	+25.49%
Application Startup	13	7	-46.15%

Table 7.15: Overall LOC Comparison Between Blazor and Vue

The overall results show that while the HTML markup requirements are similar between the two frameworks, there is a notable difference in the application logic implementation. Vue consistently requires approximately 20-30% more lines of code across the presentation logic, infrastructure, and domain layers. Although this difference may not be significant for a relatively simple application, it could lead to increased development effort and maintenance challenges in larger, more complex projects.

A key factor contributing to this difference is that Blazor benefits from using C#, which often allows for more concise code compared to TypeScript. Although Blazor applications come with a larger initial download size due to .NET assemblies, these assemblies provide features not available in TypeScript, enabling developers to implement the same functionality with fewer lines of code.

The application startup layer is the only area where Vue demonstrates an advantage. This difference can be attributed to the architectural distinctions between the frameworks. Blazor's startup code configures its dependency injection container, which, while requiring more initial setup, helps eliminate repetitive service configuration code throughout the application.

7.5. Development server startup time

This analysis was conducted to evaluate the development experience by comparing the startup times of the development servers for Blazor and Vue. To ensure a consistent and automated evaluation, a Bash script was developed. The script records a timestamp before executing the command to start the development server. Then it streams the command output to a file and repeatedly checks the logs at 0.1 second intervals for a message indicating that the server has successfully started. Once this log entry is identified, a second timestamp is recorded and the startup duration is calculated as the difference between the two timestamps. Using this script, ten measurements were taken for each framework. This approach provides an approximate measurement of the startup times of the development server. The results are presented in Table7.16.

Run	Blazor	Vue
1	12123	15322
2	9423	2345
3	9813	2270
4	10528	2063
5	11482	2092
6	9801	2112
7	11340	2059
8	10858	2043
9	9307	2020
10	8889	2078

Table 7.16: Development server startup times (ms) for Blazor and Vue

The results show that Vue's development server starts significantly faster than Blazor's after the initial run, with a median startup time of just 2.08 seconds compared to Blazor's 10.16 seconds. The exception is Vue's first run, which takes 15.3 seconds due to initial dependency resolution, pre-bundling operations, and cache creation. Once these tasks are completed, Vue's startup performance stabilizes at a consistently faster level for all subsequent runs. The performance gap is primarily due to Vue's use of Vite as its build tool and development server. Vite employs an unbundled development approach, leveraging native ES modules in the browser to compile only the necessary files on demand rather than processing the entire application at startup.

Blazor, on the other hand, must initialize the complete .NET runtime environment before the application can start. Additionally, Razor components must be compiled into .NET assemblies, introducing further overhead. Unlike Vue, which runs directly in the browser using JavaScript, Blazor requires a more complex initialization process, which results in longer startup times.

8. CONCLUSION

This study examined the suitability of using Blazor technology for the implementation of Progressive Web Applications and compared it with a JavaScript-based framework, Vue.js. To achieve this, the research focused on two key questions: the technical capabilities of Blazor in meeting PWA requirements and the advantages and disadvantages of using Blazor in comparison to JavaScript-based technologies.

The findings of this study indicate that Blazor technology enables the development of Progressive Web Applications through its WebAssembly-based implementation, which runs entirely within the browser. Essential PWA features including installability, offline access, and background synchronization, were successfully implemented in both applications developed during this research. Although Blazor primarily supports C# development, it provides interfaces to execute JavaScript code when necessary, such as requesting notification permissions.

While Blazor includes a default PWA template with basic service worker implementation for offline access, this research evaluated Blazor's compatibility with the widely adopted Workbox library, which simplifies service worker development. The integration process revealed differences between frameworks: Vue.js offers a dedicated plugin with native Workbox compatibility, enabling automatic service worker generation, whereas Blazor requires manual integration by importing Workbox from a Content Delivery Network and utilizing Workbox CLI for precaching implementation. Despite these procedural differences, the service worker was successfully implemented within the Blazor framework.

Beyond the implementation of the service worker, other aspects of Blazor must be evaluated when considering its suitability for PWA development. The most significant challenge of using this technology is its performance. Blazor WebAssembly applications require downloading the .NET runtime and necessary dependencies before execution, resulting in a larger application payload. This leads to longer startup times compared to JavaScript frameworks such as Vue.js, which are optimized for fast initial rendering. The delay is especially noticeable on lower-end devices or under poor network conditions, where extended loading times can negatively impact the user experience.

In addition to load-time concerns, Blazor WebAssembly presents further runtime performance considerations. Although it enables the execution of C# code directly in the browser, WebAssembly cannot directly manipulate the DOM. Instead, UI changes must be captured by JavaScript, passed to WebAssembly where the C# code is executed, and then returned to JavaScript. This architectural approach introduces overhead compared to Vue's direct DOM manipulation. However, in pure computational tasks, such as sorting datasets analyzed in this research, Blazor gains an advantage as the size of the dataset increases.

The difference in performance is also noticeable during the development process. Blazor's server startup is more time consuming than Vue's, due to the need to initialize the .NET runtime, compile Razor components, and configure dependency injection. In contrast, Vue benefits from Vite's lightweight, module-based compilation process, enabling faster startup and a more responsive development workflow.

The main advantage of using Blazor for PWA development is its ability to significantly reduce overall code volume, resulting in a simpler and more maintainable codebase. Using C# for application logic allows developers to write more concise code. The built-in dependency injection framework simplifies service configuration by eliminating redundancy, and the convention-based routing system automatically generates routes from the project structure, eliminating the need for separate routing configuration files. In the application developed for this research, these features reduced the number of code lines by 20–30% compared to Vue.

Based on the findings in this study, Blazor WebAssembly is a suitable option for PWA development in scenarios where performance is not the primary concern. Blazor fully meets all PWA requirements, providing features such as offline support, background sync, and installability. Although its larger application size and DOM updates across the WebAssembly-JavaScript boundary impact performance sensitive applications, it offers significant advantages in development efficiency and maintainability. It is especially suitable for teams already experienced with .NET, allowing developers to work within a familiar environment and reducing the learning curve. Using C# for front-end applications simplifies code maintenance and ensures better long-term support, making it practical for in-house applications where rapid development and ease of maintenance take priority over performance requirements.

Future studies could explore several important areas to deepen the understanding of Blazor's potential in PWA development. Although this study compared the volume of code between Blazor and a Vue.js for a relatively simple application, examining larger and more complex projects could offer greater insight into how each technology manages scalability, maintainability, and project structure.

Another valuable direction for future research involves the security aspects of Blazor PWAs, with a particular focus on user authentication and authorization. This could include evaluating Blazor's support for modern protocols such as OAuth and OpenID Connect, and comparing these implementations with those used in JavaScript-based PWAs. Additionally, it would be beneficial to assess how Blazor handles secure token management, permission scoping, and session persistence, especially in offline scenarios.

Furthermore, with the release of .NET 8, Blazor introduced an interactive auto rendering mode designed to improve initial load times by starting with server-side rendering and transitioning to WebAssembly once the necessary client-side resources are loaded. Although this approach currently lacks support for PWA development, future enhancements may introduce such capabilities. If PWA support is introduced in the future, it would provide an important area for research, especially to understand its impact on startup performance and how mixed rendering modes work with key PWA features.

REFERENCES

[1] ScandiPWA, *History of progressive web apps*, Accessed: 2024-07-09. https://medium.com/progressivewebapps/history-of-progressive-web-apps-4c912533a531, 2019.

[2] Web.dev, *Flipkart triples time-on-site with progressive web app*, Accessed: 2024-07-09. https://web.dev/case-studies/flipkart, 2016.

[3] A. Mhatre and S. Mali, "Progressive web applications, a new way for faster testing of mobile application products," *2023 3rd Asian Conference on Innovation in Technology (ASIANCON)*, 2023, pp. 1–6.

[4] Microsoft, *Asp.net core blazor hosting models*, Accessed: 2024-07-09. https://learn.microsoft.com/en-us/aspnet/core/blazor/hosting-models?view=aspnetcore-8.0&source=recommendations, 2024.

[5] M. Lang, "Webassembly approach to client-side web development using blazor framework," Master's thesis, Czech Technical University in Prague, Faculty of Information Technology, 2019.

[6] D. Suryś, P. Szłapa, and M. Skublewska-Paszkowska, "Webassembly as an alternative solution for javascript in developing modern web applications," *Journal of Computer Sciences Institute*, vol. 13, pp. 332–338, Dec. 2019.

[7] R. Ollila, N. Mäkitalo, and T. Mikkonen, "Modern web frameworks: A comparison of rendering performance," *Journal of Web Engineering*, vol. 21, no. 03, 2022. [Online]. Available: https://journals.riverpublishers.com/index.php/JWE/article/view/7217.

[8] K. Kozak and J. Smołka, "Analysis of the blazor framework in client-hosted mode," *Journal of Computer Sciences Institute*, vol. 16, pp. 269–273, Sep. 2020.

[9] Web.dev, *Progressive web apps*, Accessed: 2024-08-11. https://web.dev/learn/pwa/progressive-web-apps, 2021.

[10] J. Grigsby, *Progressive Web Apps*. Jeffrey Zeldman, 2018.

[11] S. R. Pete LePage, *What makes a good progressive web app?* Accessed: 2024-08-12. https://web.dev/articles/pwa-checklist.

[12] MDN Web Docs, *Making pwas installable*, Accessed: 2024-08-13. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable.

[13] MDN Web Docs, *Service worker api*, Accessed: 2024-08-13. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API.

[14] Web.dev, *Service workers*, Accessed: 2024-08-13. https://web.dev/learn/pwa/service-workers.

[15] MDN Web Docs, *Using service workers*, Accessed: 2024-08-13. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers.

[16] Chrome for Developers, *A service worker's life*, Accessed: 2024-08-15. https://developer.chrome.com/docs/workbox/service-worker-lifecycle.

[17] Youenn Fablet, *Workers at your service*, Accessed: 2024-08-15. https://webkit.org/blog/8090/workers-at-your-service/.

[18] Web.dev, *Caching*, Accessed: 2024-08-17. https://web.dev/learn/pwa/caching.

[19] D. Sheppard, "Caching and offline functionality with service workers," *Beginning Progressive Web App Development: Creating a Native App Experience on the Web*. Berkeley, CA: Apress, 2017, pp. 45–71. [Online]. Available: https://doi.org/10.1007/978-1-4842-3090-9_4.

[20] MDN Web Docs, *How to make pwas re-engageable using notifications and push*, Accessed: 2024-08-16. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Tutorials/js13kGames/Re-engageable_Notifications_Push.

[21] Matt Gaunt, *Web push book: How push works*, Accessed: 2024-08-16. https://web-push-book.gauntface.com/how-push-works/.

[22] C. Rojas, "Background sync," *Building Progressive Web Applications with Vue.js : Reliable, Fast, and Engaging Apps with Vue.js*. Berkeley, CA: Apress, 2020, pp. 163–186. [Online]. Available: https://doi.org/10.1007/978-1-4842-5334-2_7.

[23] Jake Archibald, *Introducing background sync*, Accessed: 2024-08-16. https://developer.chrome.com/blog/background-sync?hl=en.

[24] Microsoft Learn, *Asp.net core blazor*, Accessed: 2024-08-26. https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-8.0.

[25] C. Sainty, *Blazor in Action*. Manning, 2022. [Online]. Available: https://books.google.pl/books?id=zTRxEAAAQBAJ.

[26] Microsoft Learn, *What's new in asp.net core 8.0*, Accessed: 2024-08-26. https://learn.microsoft.com/en-us/aspnet/core/release-notes/aspnetcore-8.0?view=aspnetcore-8.0.

[27] MDN Web Docs, *Webassembly concepts*, Accessed: 2024-08-16. https://developer.mozilla.org/en-US/docs/WebAssembly/Concepts.

[28] Sebastian Gingter, *Understanding and controlling the blazor webassembly startup process*, Accessed: 2024-08-26. https://www.thinktecture.com/en/blazor/understanding-and-controlling-the-blazor-webassembly-startup-process/.

[29] Blazor University, *Render trees*, Accessed: 2025-03-12. https://blazor-university.com/components/render-trees/.

[30] Vue.js, *Introduction*, Accessed: 2024-09-20. https://vuejs.org/guide/introduction.html.

[31] Vue.js, *Using vue with typescript*, Accessed: 2024-09-20. https://vuejs.org/guide/typescript/overview.html.

[32] Vue.js, *Rendering mechanism*, Accessed: 2024-09-20. https://vuejs.org/guide/extras/rendering-mechanism.

[33] Vite, *Why vite*, Accessed: 2024-09-20. https://vite.dev/guide/why.html.

[34] Chrome for Developers, *Workbox-precaching*, Accessed: 2024-09-25. https://developer.chrome.com/docs/workbox/modules/workbox-precaching.

[35] Firebase, *Firebase hosting*, Accessed: 2024-09-25. https://firebase.google.com/docs/hosting.

[36] Chrome for Developers, *First contentful paint*, Accessed: 2024-09-25. https://developer.chrome.com/docs/lighthouse/performance/lighthouse-largest-contentful-paint.

[37] Chrome for Developers, *Largest contentful paint*, Accessed: 2024-09-25. https://developer.chrome.com/docs/lighthouse/performance/lighthouse-largest-contentful-paint.

[38] Chrome for Developers, *Total blocking time*, Accessed: 2024-09-25. https://developer.chrome.com/docs/lighthouse/performance/lighthouse-total-blocking-time.

[39] Chrome for Developers, *Speed index*, Accessed: 2024-09-25. https://developer.chrome.com/docs/lighthouse/performance/speed-index.

[40] Microsoft Learn, *Primary constructors*, Accessed: 2025-03-01. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/proposals/csharp-12.0/primary-constructors.

LIST OF FIGURES

3.1 The lifecycle of service worker 15

6.1 Architecture of applications 23
6.2 Component structure of applications 24
6.3 Desktop design 26
6.4 Mobile Design 27
6.5 Install dialog on mobile platform 30

7.1 Cache Storage tab of Blazor PWA 40
7.2 Cache Storage tab of Vue PWA 40

LIST OF TABLES

7.1 Blazor results on desktop device(Clear Storage enabled) 37
7.2 Blazor results on mobile device(Clear Storage enabled) 37
7.3 Vue results on desktop device(Clear Storage enabled) 38
7.4 Vue results on mobile device(Clear Storage enabled) 38
7.5 Blazor results on desktop device(Clear Storage disabled) 38
7.6 Blazor results on mobile device(Clear Storage disabled) 38
7.7 Vue results on desktop device(Clear Storage disabled) 38
7.8 Vue results on mobile device(Clear Storage disabled) 39
7.9 Cache Sizes for Blazor and Vue PWAs 40
7.10 Detailed execution times (in milliseconds) for Blazor and Vue 42
7.11 Lines of code comparison between Blazor and Vue PWA 43
7.12 Comparison of LOC in Player Service Methods 44
7.13 Comparison of LOC, Characters, and Characters per line 45
7.14 LOC in Application Startup Files 46
7.15 Overall LOC Comparison Between Blazor and Vue 46
7.16 Development server startup times (ms) for Blazor and Vue 47